Session 3 speaking script: Compile-Time and Generic Programming

How to use this script

- **Bold lines** are the must-say sentences. If you say nothing else on a slide, say those.
- Plain paragraphs are the talk track, written to be read aloud. Paraphrase freely once you have rehearsed.
- `(pause)` and `(beat)` are deliberate stops. A pause is two full seconds of silence; a beat is one. You are a fast speaker: these are not optional.
- `>> DO:` lines are actions (terminal, editor, Compiler Explorer). `>> ASK:` lines are questions to the room, with the answer you expect and what to say after.
- `>> IF AHEAD:` is 60 to 120 seconds of extra depth to use instead of speeding up. `>> IF BEHIND:` is the one sentence to say instead of the talk track.
- Target clock times assume the session starts at 0:00 when you put the title slide up.

Pace plan for a fast speaker

Four blocks: about twenty minutes of slides, then a ten-minute lab, four times. The lab is the checkpoint; if the lecture runs long, the lab still starts on time and the slides you did not reach are covered in the lab debrief or skipped (they are all in the deck's Further reading appendix for the PDF). Block 2 is scripted for fifteen minutes, so its lab starts at 0:45 and the five minutes before block 3 at 1:00 are the buffer for the first half running long.

Checkpoint	Target clock
1. Title	0:00
5. `constexpr` in C++11 (block 1, `constexpr`)	0:06
11. `consteval`	0:17
13. Lab 3.1 launched	0:20
14. The problem with C++11 templates (block 2, C++17 templates)	0:30
22. Lab 3.2 launched	0:45
Buffer: block 3 starts on the clock even if block 1 ran long	0:55 to 1:00
23. The problem concepts solve (block 3, Concepts)	1:00
29. Subsumption	1:10
35. Lab 3.3 launched	1:20
36. Three problems, one feature (block 4, Deducing `this` and the diagnostics)	1:30
42. Reading the diagnostics	1:43

Checkpoint	Target clock
43. Lab 3.4 launched	1:45
44. Exercise (at home), 45. Takeaway	1:55
Done	1:58

If you hit a lab launch more than 2 minutes early, use the IF AHEAD material in the slide before it rather than starting the lab early; people need the clock to be predictable. If you are late, cut the slide before the takeaway, never the lab.

Slow down deliberately on these four:

- **Slide 7, Where `constexpr` values live.** "May" versus "must" is the single most common misunderstanding of `constexpr`, and everything after it (`consteval`, and lab 3.1's TODO 2) depends on it landing.
- **Slide 11, `consteval`.** The must half. Lab 3.1 has people make a validator `consteval` and then be surprised that a call in `main` still compiles; set that up here.
- **Slides 29 and 30, Subsumption.** The rule that only named concepts subsume is counterintuitive, and it is the reason lab 3.3's `SerializableRange` and the exercise's are written the way they are.
- **Slide 37, Explicit object parameter.** Forwarding references plus value categories, applied to `*this`. Half the room is still fuzzy on `Self&&` from Session 2's supplemental deck, and lab 3.4's TODO 3 is exactly this.

Before class checklist

- Build with GCC 14 and run the tests, labs included: `cmake -S . -B build -G Ninja && cmake --build build && ctest --test-dir build -R "s03|lab_s03" --output-on-failure`. Everything should pass. Binaries land at `./build/demos/s03/demo_s03_<stem>`.
- Open the four lab starters on Compiler Explorer in four tabs, from the links on slides 13, 22, 35 and 43, and do each lab yourself once: `lab1_constexpr`, `lab2_templates17`, `lab3_concepts`, `lab4_deducing_this`. Keep the four solution links ready to paste at the seven-minute mark of each lab. For lab 3.3, also run the `to_json(Opaque{1})` line before and after the constraint so you have both error messages in your head.
- For the `SHOW_ERRORS` demos, have a compile line ready in the terminal history, for example `g++-14 -std=c++23 -fsyntax-only -DSHOW_ERRORS demos/s03/concepts_vs_sfinae.cpp`. If you build with Clang, use your `clang++ -std=c++23 -stdlib=libc++` equivalent.
- Demo files to have open in the right-hand editor group, in deck order: `constexpr_evolution.cpp`, `constexpr_alloc.cpp`, `consteval_constinit.cpp`, `fold_expressions.cpp`, `if_constexpr_dispatch.cpp`, `variable_templates.cpp`, `ctad_guides.cpp`, `auto_nttp.cpp`, `concepts_basics.cpp`, `requires_expressions.cpp`, `subsumption.cpp`, `concepts_vs_sfinae.cpp`, `deducing_this.cpp`. (The appendix slides' demos, `constexpr_limits.cpp`, `is_constant_evaluated.cpp` and `template_lambdas.cpp`, only if you expect to use the IF AHEAD material.)

- Compiler Explorer tabs to preload besides the labs (the demo headers still say `<add short link>`, so build these by hand and save the short links):
  1. The solution's `make_crc_table` and `crc16` (from `exercises/s03-compile-time/solution/include/telemetry/crc.h`), GCC 14, `-std=c++23 -O2`, with a tiny `int main` that calls `crc16` on a literal. You want to show `kCrcTable` as a data symbol and the call folded to an immediate.
  2. Two panes, same call `serialize(ParseError::EmptyLine)`: left pane with the starter's `serialize.h` inlined, right pane with the solution's. GCC 14 in both. This is slide 23 and slide 42.
  3. `concepts_vs_sfinae.cpp` with `-DSHOW_ERRORS`, one GCC 14 pane and one Clang 18 pane.
  4. Optional, for a question about appendix slide 51: the `validate(kRpm)` snippet with `static_assert(validate(kRpm).empty(), validate(kRpm));` on Clang 18 with `-std=c++2c`. Clang prints the returned reason as the message.
- Slide 42's table break: find which sensor in the solution's `config.h` actually breaks when you set `.min_valid = 500.0` (it must be one whose `max_valid` is at or below 500; `rpm` with a 12000 max will not break). Note the exact edit.
- Know what the broken-table error actually prints. Verified on GCC 13 and Clang 18: you get the `static_assert` message and the failing expression, **not** the string your `validate` returned. Do not promise the room they will see the reason text. Lab 3.1's TODO 2 shows the same thing.
- Have the exercise README ready to paste for the at-home launch: `exercises/s03-compile-time/README.md`. Confirm `ctest --test-dir build -R s03_report_identical` passes on the starter as-is.

The script

1. The Evolution of C++: Session 3 · target 0:00, ~1 min

Welcome back. This is session three, and it's the session where we make the compiler do our work.

Today has two halves: making the compiler compute, and making the compiler check. The first half is `constexpr` and its relatives, `consteval` and `constinit`. The second half is concepts. In between there's a short bridge of C++17 template features that made templates readable, and at the end there's one C++23 feature, deducing `this`, that takes out the last of the template boilerplate.

(beat)

Same telemetry program as the last two sessions. The exercise has one target for each half, plus deducing `this`. Everything you see today ends up in that code.

Bridge: here's how the two hours break down.

2. Agenda · target 0:01, ~1 min

Four blocks. **Twenty minutes of me, ten minutes of you, four times.** Block one is `constexpr` and `consteval`: making the compiler compute. Block two is the C++17 template features that made templates readable: folds, `if constexpr`, CTAD. It is the short block; its spare five minutes are the buffer if block one runs long. Block three is concepts: making the compiler check. Block four is deducing `this`, and reading the diagnostics.

(beat)

Each lab is one file on Compiler Explorer with three TODOs. The slide that launches it has the link; you need nothing installed. If you built the repo, the same files are under `labs/`.

The telemetry exercise is at-home work this time. Its first three tasks are today's labs applied to the real program, so you will have done them once already. **The exercise starter is the Session 2 solution, so if your Session 2 build works, today's will too.**

(pause)

Bridge: first, where we left the program last time.

3. Session 2 recap · target 0:02, ~2 min

Last session's solution: an `expected` parser, `optional` lookups, `span` parameters, `string_view` fields, and a `std::print` report with three formatters. Same report, byte for byte. That byte-for-byte test is still in force today. We are going to change a lot of the code and none of the output.

Three places people got stuck, and they're worth a minute because they'll come back.

First: forgetting `std::unexpected` around the error. If your function returns `expected<Record, ParseError>`, a bare `return ParseError::EmptyLine;` doesn't convert. That's deliberate. The designers didn't want an implicit conversion from the error type, because if T and E were ever convertible to each other you'd silently return the wrong alternative. **`unexpected` is the annotation that says "this is the failure path", and you want that visible in review.**

Second: `find_sensor(name).transform(&SensorConfig::units)`. That looks like it should work. It doesn't, on either compiler. `find_sensor` returns an optional by value, so it's a temporary. Calling a pointer-to-member on an rvalue gives you an rvalue reference, and `optional` can't hold a reference. A lambda that returns by value fixes it. That's a value-category fact, and the supplemental deck on value categories walks through it.

(beat)

Third: a `formatter::format` that isn't a template on the context type. libstdc++ accepted it. libc++ rejected it, because libc++ checks the formatter against a compile-time context type. **If you only build on one toolchain, you only find out about this kind of thing when somebody else builds your code.** That's an argument for building on two compilers in CI, and it's why this repo does.

ASK: "Who hit at least one of those three?" Expect most hands for the first or second. Say: "Good. The second one is exactly the kind of thing concepts make readable, and we'll see the error-message side of that today."

IF BEHIND: "The three stumbles were `unexpected`, the `transform` on a temporary, and the formatter template; all three are in the solution's comments."

Bridge: so why spend a whole session on compile time? Because of three specific lines in the starter.

4. Why compile time matters here · target 0:04, ~2 min

Three things in the starter do work at runtime that never changes. They were put there for today.

Row one: `crc.cpp` builds a 256-entry CRC lookup table behind a function-local static. That means the first call pays the latency of building it, and every call after that pays a guard check: the compiler inserts a thread-safe "has this been initialized yet" test. Small, but it's on every checksum. **The table never changes. There is no reason for the target to compute it at all.** After today it's `inline constexpr auto kCrcTable = make_crc_table();`, it's in read-only data, and there's no code that builds it.

Row two: `config.cpp` holds the sensor configuration table. Nothing checks it. If somebody types a minimum that's bigger than the maximum, or duplicates a sensor name, that ships. It doesn't crash. It misbehaves: every reading gets flagged out of range, or the wrong sensor's limits get used. That's the worst kind of bug in a fielded system, because it looks like a sensor problem. After today, `static_assert(validate(kSensors).empty())` means a bad table doesn't build.

(pause)

Row three: `serialize.h` is a pile of overloads. Hand it a type it doesn't support and you get a page of candidates. After today it's a handful of constrained templates and a `Serializable` concept, and the error names what's missing.

Zero runtime cost, no static-initialization order, and one class of bug that cannot reach a running program. That last phrase is the one I care about. In a codebase that lives for twenty years, under a coding standard, through code review, the cheapest defect is the one the build refuses.

The last line on the slide is the shape of the day. **Making the compiler compute, that's `constexpr` and `consteval`; making the compiler check, that's concepts.** C++17 templates are the bridge between the two, and deducing `this` removes the last of the boilerplate. Keep "may, must" in your head for block one; that's the whole vocabulary.

ASK: "Where in your own code is there a table that's built at startup and never changes?" Expect: CRC tables, lookup tables for scaling or calibration, message ID maps, register maps. Say: "Every one of those is a candidate for the first half of today."

IF BEHIND: "Three rows, three exercise tasks: the CRC table, the config table, and `serialize`."

Bridge: let's start where `constexpr` started, which was not very impressive.

5. `constexpr` in C++11: where it started · C++11 · target 0:06, ~1 min

`constexpr` in C++11 was a function that could run at compile time, as long as its body was a single return statement. That's it.

So the factorial on the slide is a ternary that calls itself. No loops: loops are spelled as recursion. No local variables. No `if` statements. No mutation. **C++11 `constexpr` was a toy, and it was useful for array bounds and little else.** If you've seen C++11-era code with constexpr functions that look like Lisp, that's why.

(beat)

It's on screen so the rest of the story has a starting point. Everything after this slide is the committee removing restrictions, one standard at a time.

> IF AHEAD: One C++11 detail that still bites in old code: in C++11 a `constexpr` member function was implicitly `const`. So if you're maintaining a C++11 codebase and you add `constexpr` to a setter, it silently becomes a `const` member function and stops compiling where it mutates. That changed in C++14, which is the next slide. Also, recursion depth is bounded: GCC's default `-fconstexpr-depth` is 512, Clang's is 512 too, so a recursive C++11-style function on a large input fails to evaluate at compile time and, if the result was required as a constant, fails the build.

Bridge: C++14 made it look like a normal function.

6. Relaxed `constexpr` · C++14 · target 0:07, ~2 min

Left: the C++11 form. Right: C++14. Look at the right side: a local `int r = 1`, a `for` loop, `r *= i`, return. **From C++14 on, a `constexpr` function looks like a normal function with a keyword in front.** Loops, locals, `if`, mutation of locals, multiple return statements.

Why does this matter for us? Because the CRC table loop in the exercise is a nested loop with a local that gets shifted and XORed. That's legal from C++14. You don't need C++20 for the first exercise task; you need C++14 plus C++17's `std::array`, which is coming up.

(pause)

The other C++14 change is at the bottom of the slide: `constexpr` member functions are no longer implicitly `const`. So you can write a `constexpr` setter, or a `constexpr` builder that mutates `*this`. That matters at the end of today, when `SensorStats::add` becomes `constexpr` and we evaluate a three-sample accumulator in a `static_assert`.

> DO: In the right-hand editor, scroll `demos/s03/constexpr_evolution.cpp` to the bottom and point at the `static_assert` lines: `factorial11(5) == 120`, `factorial14(5) == 120`. Say "these are the tests; if this file compiles, they passed."
>
> ASK: "If the C++11 and C++14 versions both give 120, why would you ever care which one you wrote?" Expect: readability, recursion depth limits, maintainability. Say:

"Right. And in a coding standard that bans recursion, which several embedded ones do, the C++11 form wasn't even allowed."

IF AHEAD: MISRA-style rules that ban recursion (MISRA C has one; MISRA C++ has a similar rule) made C++11 `constexpr` nearly unusable in those shops, because every loop had to be recursion. Relaxed `constexpr` is what made `constexpr` compatible with that kind of coding standard. Also worth knowing: the compiler caps how much work a constant evaluation can do. GCC has `-fconstexpr-loop-limit` and `-fconstexpr-ops-limit`; Clang has `-fconstexpr-steps`. A 256-entry table is nowhere near the limits. A million-entry table might be.

IF BEHIND: "C++14 lets `constexpr` functions have loops and locals; that's what makes the CRC loop legal."

Bridge: before going further, where do these values actually live, and when does the compiler actually run the function?

7. Where `constexpr` values live · target 0:09, ~2 min (slow down)

This slide is the one I most want you to get right, so I'm going to slow down.

Top two lines are variables. `constexpr int kMax = 256;` is a constant. `constexpr` on a variable implies `const`, and it means the initializer must be a constant expression. Second line: `inline constexpr auto kTable = make();` That's C++17. The `inline` matters in a header. A namespace-scope `const` variable has internal linkage, so without `inline` every translation unit that includes the header gets its own copy, with its own address. `inline` says there's one entity across the program. **For a constant table in a header, write `inline constexpr`.**

(pause)

Now the functions. Look at the third line: `constexpr int f(int n)`. The comment says MAY run at compile time. Not must. May.

Line four: `int a = f(3);` The variable `a` isn't `constexpr`, so nothing requires a constant. At block scope, that's a runtime call. The optimizer will probably fold it at `-O2`, but that's the optimizer being nice, not the language promising anything. At `-O0` you'll see a call instruction.

Line five: `constexpr int b = f(3);` Now `b` must be a constant, so `f(3)` must be evaluated at compile time. Guaranteed.

Line six: `static_assert(f(3) == 6);` Also compile time, and it's a unit test.

(pause)

A `constexpr` function is one function: the same body serves both contexts. It runs at compile time only when a constant is required: a `constexpr` variable, a template argument, an array bound, a `static_assert`. Otherwise it's an ordinary function call.

`constexpr` on a function permits compile-time evaluation. It does not force it. If you take one sentence out of this block, take that one. People put `constexpr` on a function, call it

in a hot loop with a runtime argument, and believe they've moved the work to compile time. They haven't. The keyword that forces it is `consteval`, four slides from now.

> ASK: "Line four, `int a = f(3);`. Compile time or runtime?" (pause) Expect a split room, some say compile time because the argument is a literal. Say: "Language answer: runtime. Optimizer answer: probably folded. The only way to get a guarantee is to require a constant, which is line five."
>
> IF AHEAD: There's a wrinkle at namespace scope. If `int a = f(3);` is a global, not a local, the rules for static initialization say the compiler must constant-initialize it if the initializer is a constant expression. So the same line is "runtime" at block scope and "constant-initialized" at namespace scope, and nothing in the code tells you which you got. If someone later changes `f` so it can't be constant-evaluated, the global silently becomes dynamically initialized, and now it's subject to initialization order across translation units. That's precisely the problem `constinit` solves; it is in the appendix, slide 52. Also: `constexpr` functions are implicitly `inline`, so they live in headers, and their bodies are visible to every caller. That's a header-hygiene cost you should know about in a large build.
>
> IF BEHIND: "`constexpr` means may, not must; it's compile time only where a constant is required."

Bridge: C++17 brought lambdas into constant expressions.

8. `constexpr` lambdas and `if constexpr` · C++17 · target 0:11, ~2 min

C++17 adds two things to the `constexpr` story. Lambdas can be `constexpr`, and `if constexpr` can pick a branch at compile time. I'll cover `if constexpr` in block two; it's mostly a template tool.

Look at the code. `factorials17` returns a `std::array<int, 6>`. Inside, there's a local array, a lambda called `fact` that's marked `constexpr`, and a loop that fills the array. At the bottom, `inline constexpr auto kFactorials = factorials17();` That's the pattern you'll use in the exercise: **a `constexpr` function that builds a table, and an `inline constexpr` variable that holds the result, computed once, by the compiler, in the header.**

(beat)

The `constexpr` on the lambda is optional. Since C++17, lambdas are implicitly `constexpr` whenever their body qualifies. Writing it explicitly does two things: it tells the reader you intend compile-time use, and it makes the compiler error out if somebody later adds something to the body that can't be constant-evaluated. That's the same reason you'd write `override`: it's a checked statement of intent.

Note the `static_cast<std::size_t>(i)` on the index. That's there because the repo builds with `-Wall -Wextra -Wpedantic -Werror`, and sign-conversion warnings are the kind of thing a strict build catches. You'll see that pattern in the solution.

> IF AHEAD: An explicitly `constexpr` lambda whose body can't be constant-evaluated is an error at the point of definition only if no invocation could ever be constant; in

practice compilers diagnose obvious cases like calling a non-`constexpr` function. The implicit rule is what makes generic algorithms work in constant expressions: a C++20 `std::sort` with a lambda comparator runs at compile time without anyone marking the lambda. That's slide 10, two slides from now.

IF BEHIND: "Lambdas are `constexpr` when they can be; write the keyword to make intent checked."

Bridge: here's the exercise's version of that pattern.

9. `std::array` at compile time · C++17 · target 0:13, ~2 min

This is exercise task one, from the solution's `crc.h`. Hand-typed excerpt.

`make_crc_table` returns `std::array<std::uint16_t, 256>`. It's the same loop as the starter's `CrcTable` constructor. Outer loop over 256 byte values. Start with `i << 8`. Inner loop, eight bits: if the top bit is set, shift left and XOR the polynomial; otherwise just shift. Store it.

Then line nine: `inline constexpr auto kCrcTable = make_crc_table();` The comment says 512 bytes in `.rodata`, no startup code. 256 entries times two bytes.

(pause)

Line ten is the one I like. `static_assert(crc16("123456789") == 0x29B1);` That string is the standard check input for CRC algorithms, and `0x29B1` is the published check value for this CRC-16 variant. **That `static_assert` is a unit test that runs on every build, costs nothing at runtime, and can't be skipped by someone who forgets to run the test suite.**

Why does this matter on embedded targets? `.rodata` on a microcontroller usually lives in flash. A table built at runtime lives in RAM, and the code that builds it lives in flash too. So the compile-time version saves RAM and code, and it's there before `main`.

DO: Switch to the Compiler Explorer tab for the CRC table (tab 1). GCC 14, `-std=c++23 -O2`. Point at the assembly: `kCrcTable` appears as a data label with `.value` (or `.short`) directives, 256 of them. Then point at `main`: the call to `crc16` on a literal has folded to a `mov` of an immediate (`0x29B1`, decimal 10673). Say "No loop. No table construction. The answer is in the instruction." Recovery if the output is noisy or the tab didn't load: "The point is: the table is data, not code. You'll see it yourself in the exercise; the `static_assert` proves the value."

ASK: "What did we lose by doing this?" (pause) Expect: compile time, or "nothing". Say: "A little compile time, which for 256 entries is unmeasurable. And one thing people miss: the table is now in the header, so changing the polynomial recompiles everything that includes it. In a big build, that's a real cost, and it's a reason to keep tables like this in their own small header."

IF AHEAD: Why does `crc16` of a literal fold at `-O2` even though `crc16` is called in a non-constant context? Because the optimizer can see the whole body and all inputs. That's optimization, not a guarantee. If you need the guarantee, assign it to a `constexpr` variable. A related review heuristic: if a checksum of a constant appears

in a protocol header, write it as `constexpr auto kHeaderCrc = crc16(...)` so the reviewer can see it's compile-time by construction.

IF BEHIND: "The CRC table becomes `inline constexpr`; the `static_assert` on `0x29B1` is the unit test."

IF AHEAD: What a constant expression refuses, appendix slides 48 and 49. Undefined behavior is not allowed in one: signed overflow, out-of-bounds, an uninitialized read, all hard errors during constant evaluation, so the compiler is a sanitizer for every `constexpr` call it evaluates, at no runtime cost. And `reinterpret_cast` is never allowed, which is why `std::as_bytes` is not `constexpr` and why the at-home exercise's task 1 has a snag in it. Both slides are in the PDF with the demo file `constexpr_limits.cpp`.

Bridge: C++20 made the biggest jump. You can allocate.

10. Dynamic allocation at compile time · C++20 · target 0:15, ~2 min

C++20 lets you use `new` and `delete` in a constant expression, which means `std::vector` and `std::string` work at compile time. With one rule.

Look at `sorted_names`. A `std::vector<std::string_view>` with four sensor names. `std::sort` on it: the standard algorithms are `constexpr` in C++20. Copy into a `std::array`. Return the array. The vector is destroyed at the end of the function.

(pause)

The allocation must be freed before the constant expression ends. That's called transient allocation. The vector is scratch space inside the computation. Only the `std::array` escapes. And the `static_assert` says the first sorted name is `current_bus`.

The commented line at the bottom: `constexpr std::vector<int> kNotAllowed = {1, 2, 3};` That's an error. A `constexpr` vector variable would need its heap allocation to survive into the running program, and there's no heap at compile time to hand over. Proposals exist to allow that; it isn't in the language you can use today.

So the pattern is: **build with `vector`, return an `array`.** If you need the size, compute it in one `constexpr` call and use it as the array bound in another.

DO: Run `./build/demos/s03/demo_s03_constexpr_alloc`. Expected output: `temp_core 35`. Point at the right-hand file, the `joined_length` function below the snippet: it builds a `std::string` at compile time and returns only its size, and the `static_assert` checks 35. Say "the string is transient, its size isn't." Recovery: "The `static_assert`s are the demo; if it compiled, it worked."

IF AHEAD: The "compute the size first" trick is common enough to name. You call the builder once to get `.size()` as a `constexpr std::size_t`, then call it again to fill a `std::array<T, N>` of exactly that size. It's two evaluations, which costs compile time, but it gives you exactly-sized tables from variable-length logic. Library support: constexpr `vector` and `string` arrived in GCC 12's libstdc++ and in libc++ around 15, so older certified toolchains won't have this even with `-std=c++20`.

IF BEHIND: "C++20 allocates at compile time as long as the memory is freed before the expression ends; build in a vector, return an array."

IF AHEAD: The rest of C++20 `constexpr` is appendix slide 50: virtual calls, `try/catch`, unions, and `std::is_constant_evaluated()`, which lets one function be exact at compile time and fast at runtime. The trap on that slide is worth one sentence here: `is_constant_evaluated()` inside `if constexpr` is always true, because the condition is itself constant-evaluated. C++23's `if consteval` cannot be misused that way.

Bridge: if `constexpr` means may, what's the keyword for must?

11. `consteval`: must run at compile time · C++20 · target 0:17, ~2 min

`consteval` declares an immediate function. **Every call to a `consteval` function must produce a constant. Calling it at runtime is a compile error.**

Look at `fnv1a`. The 32-bit FNV-1a hash: start with the offset basis, `2166136261`, and for each character XOR it in and multiply by the FNV prime, `16777619`. It's `consteval`.

`constexpr auto kRpmId = fnv1a("rpm");` Fine. The argument is a literal, the result is a constant.

The commented line: a function `id(std::string_view s)` that calls `fnv1a(s)`. Error. `s` isn't a constant, so `fnv1a(s)` can't be immediately evaluated.

(pause)

Why would you want a function that refuses to run? Because some work should never cost runtime. Hashes of string literals. Table generation. Validation. With `constexpr`, somebody can call your hash with a runtime string in a hot loop and nobody notices. With `consteval`, **it can never leak into startup or a hot loop, and it can't be misused to do so.**

The classic use is switching on strings: hash the case labels at compile time with `consteval`, hash the input at runtime with a separate ordinary function, and compare integers.

The last line on the slide: **two relatives of this are in the appendix.** Slide 51 is a `consteval` validator that returns the reason it failed, which is the exercise's task 2 and lab 3.1's TODO 2 in miniature. Slide 52 is `constinit`: a mutable global whose initializer is proven constant, the end of the static initialization order fiasco. One sentence each; the PDF has the rest.

DO: Run `./build/demos/s03/demo_s03_consteval_constinit`. Expected output: `4DD8BED6 1`. Point at the hex: "That's FNV-1a of `rpm`, computed by the compiler." Then uncomment line 17 (the `id` function), rebuild with `cmake --build build --target demo_s03_consteval_constinit`, and show the error saying the call is not a constant expression because `s` is not a constant. Re-comment it. Recovery: "The error text is in the comment; the point is that the compiler refuses the runtime call."

IF AHEAD: In C++20 as originally published, `consteval` was viral in an annoying way: calling a `consteval` function from inside a `constexpr` function with a

parameter was an error, even if the `constexpr` function was only ever used at compile time. C++23 fixed this with "consteval needs to propagate up" (P2564), applied as a defect report: such a `constexpr` function is implicitly promoted to `consteval` instead. GCC 14 and recent Clang implement it. If you're on an older compiler and see "is not a constant expression" on a call that obviously is constant, that's the reason. Also: you can't take the address of a `consteval` function in runtime code, because there's no runtime function to point to.

IF BEHIND: "`consteval` must run at compile time; a runtime call doesn't compile."

Bridge: the takeaway, then the first lab.

12. `constexpr` takeaway · target 0:19, ~1 min

`constexpr` by default on any function that could be: it costs nothing and unlocks `static_assert`. `consteval` when it must never run at runtime. `constinit` for mutable globals, in the appendix.

`static_assert` is your compile-time unit test, and undefined behavior in a constant expression is a compile error.

(pause)

Monday morning: find one lookup table that's built at startup and make it `inline constexpr`. Check the map file before and after. You'll see it move from code plus RAM to read-only data.

Time check: 0:20, and lab 3.1 starts now. It is this block in one file: a table, a validator, a parser.

IF BEHIND: Read the bold lines and the Monday morning line. Start the lab on time.

After the lab: templates. A shorter block, and it starts with an error message.

13. Lab 3.1: work that belongs at compile time · target 0:20, ~10 min

Ten minutes. Three TODOs, in order. Build after each one; each TODO ends with a `static_assert` you uncomment, and if it compiles, it passed. The link is on the slide; the same file is `labs/s03/lab1_constexpr.cpp` if you built the repo.

DO: Put the lab slide up and paste the starter link into chat. Say nothing for the first minute while people get the file open. Then: "Every TODO is two keywords and one uncommented line. The compiler tells you which keyword you forgot."

What to watch for:

- TODO 1: people make `make_crc8_table` `constexpr` and leave `kCrc8` as `const`. The `static_assert` then says "the value of `kCrc8` is not usable in a constant expression", with a note "`kCrc8` was not declared `constexpr`". Both halves are needed: a `constexpr` function may run at compile time; a `constexpr` variable requires it. That is slide 7 again.
- TODO 2, first error: `validate` is `consteval` but `kSensors` is still `const`. The error lands on the call in `main`, not on the `static_assert`: "call to consteval function ... is

not a constant expression", because a `const` array of class type is not usable in a constant expression. Making `kSensors` `constexpr` fixes both lines at once.

- TODO 2, the surprise: once `kSensors` is `constexpr`, the call in `main` still compiles. People expect `consteval` to reject a call inside `main`. It does not; it rejects a call whose arguments are not constants. `kSensors.data()` is a constant, so the compiler evaluates `validate` there too and `main` prints a baked-in "yes". Say that out loud; it is the "must" half of may-versus-must.
- TODO 2, the break: duplicate an id and the error is "static assertion failed" pointing at the `static_assert` line. The returned reason string is not printed. Same as the exercise's task 2, and worth saying once.
- TODO 3: one keyword, no body change. If someone says "that was too easy", that is the point: the function did not change, the contexts it can be used in did.

At seven minutes, paste the solution link. Walk the three diffs in two minutes: the two `constexpr` keywords on TODO 1, `consteval` plus `constexpr` on TODO 2 (point at the comment in `main`: that call is evaluated by the compiler too), and the solution's `main` on TODO 3, which calls `parse_hex` on a runtime `char frame[]` to prove the runtime path still exists.

> ASK: "What runs at start-up now?" Expect: "nothing", or "just the printfs". Then: "Two tables and two checks moved into the compiler. The binary has the answers in `.rodata` and no code that computes them. That is the whole first block."

At ten minutes, move on whether or not people are done. Nothing in block 2 depends on this lab.

---

14. The problem with C++11 templates · target 0:30, ~2 min

This is a real error from the Session 2 starter. The call is `serialize(ParseError::EmptyLine)`. `ParseError` is an enum class. There's no `serialize` for it.

Read down the candidates with me. `serialize(int)`, with a note that there's no conversion from `ParseError`. `long long`. `unsigned long`. `double`. `string_view`. `Status`. `const Record&`. And a template over `std::span<const T>`, where deduction failed.

(pause)

That's nine candidates: seven overloads plus the two templates, `span` and `vector`. **Every one of them tells you what didn't match. None of them tells you what would have.** You have to read all nine and reverse-engineer the intent.

And this is the simple case. These are plain overloads. When the overloads are `enable_if` templates, each candidate's note includes a `substitution failed` with a `no type named 'type' in 'struct std::enable_if<false, int>'` and the error runs to pages. If you've ever scrolled through a Boost or Eigen error looking for the one line that matters, you know.

Keep this error in mind. It comes back at the start of block three with the C++20 version next to it.

> ASK: "Who reads template errors from the top, and who reads them from the bottom?" Expect a split. Say: "Both are coping strategies. The goal of the next two blocks is an error you can read from the top."
>
> IF BEHIND: "Eight candidates, no reason; this error comes back in block three."

Bridge: the first C++17 fix is for variadic templates.

15. Fold expressions · C++17 · target 0:32, ~3 min

Left side, C++11. Every variadic function was a recursive pair: a base case with no arguments, and a template that peels off `first`, does something with it, and recurses on `rest...`. Two functions, and the compiler instantiates one copy for every pack length.

Right side, C++17. **A fold expression applies an operator across the whole pack in one expression.**

`sum`: `return (vs + ...);` The parentheses are required. That's a unary right fold: `v1 + (v2 + v3)`.

`all_positive`: `((vs > 0) && ...)`. Any binary operator works, including &&, and it short-circuits like you'd expect.

(pause)

`join` is the important one. Look at the comma: `((out += std::to_string(vs) + ","), ...)`. The operator being folded is the comma operator. So it expands to "do this for v1, then this for v2, then this for v3". **The comma fold is "do this for each element", and it's evaluated left to right, guaranteed, because the comma operator sequences its operands.** That's exercise task five, `serialize_all`, almost verbatim.

`sum_from_100`: `(100 + ... + vs)`. A binary left fold with an initial value. You need the binary form when the pack might be empty, which is the next slide.

> DO: Run `./build/demos/s03/demo_s03_fold_expressions`. Expected output: `6 6 true 1,2, 103`. Point at `1,2,`: "trailing comma: the simple join has a separator problem. The exercise's version fixes that with a `sep` variable that starts empty and becomes `", "` after the first element." Recovery: "The output is in my notes: six, six, true, one-comma-two-comma, one-oh-three."
>
> IF AHEAD: Two code-review notes on comma folds. One: if any element type overloads `operator,`, which is rare but legal, the fold calls the overload. Defensive libraries write `(static_cast<void>(f(vs)), ...)` to force the built-in comma. Two: the exercise's fold `((out += sep, out += serialize(vs), sep = ", "), ...)` has a comma expression inside a comma fold. That's legal and readable once you see it, but it's worth one comment in the code, because the next maintainer won't recognize it.

IF BEHIND: "A fold applies an operator across a pack; the comma fold means 'for each', left to right."

Bridge: there are four forms, and one gotcha.

16. The four fold forms · C++17 · target 0:35, ~1 min

Four shapes. Pack on the left with the dots on the right is a right fold. Dots on the left is a left fold. Add an initial value and it's a binary fold.

Left versus right rarely matters for associative operators like + and &&. It matters for -, for /, and for <<. A left fold of << over `std::cout` is how you stream a pack: `(std::cout << ... << vs)`.

(beat)

The gotcha is the last line. With an empty pack, && gives `true`, `||` gives `false`, comma gives `void()`. Every other operator is an error. So `(vs + ...)` with zero arguments doesn't compile. **If the pack can be empty, use the binary form with an initial value.**

IF AHEAD: The empty-pack rule is an easy unit-test hole. A test suite that always calls `sum(1, 2, 3)` will never catch that `sum()` doesn't compile. Since it's a compile error and not a runtime one, a single `static_assert(requires { sum(); })` or just a line that calls `sum()` in a test file covers it.

IF BEHIND: "If the pack can be empty, use the binary form with an init value."

Bridge: `if constexpr` replaced the other big C++11 template trick.

17. `if constexpr` replacing dispatch · C++17 · target 0:36, ~3 min

Left, C++11. Two overloads of `describe11`, each guarded by `enable_if` on a type trait. One for integral types, one for floating point. To understand what `describe11` does, you have to find all the overloads, read each `enable_if`, and reassemble the logic in your head. If somebody adds a third overload in another header, you have to find that too.

Right, C++17. One function. `if constexpr (std::is_integral_v<T>)`, `else if constexpr (std::is_floating_point_v<T>)`, `else`. You read it top to bottom like any other function.

(pause)

The untaken branch is discarded, not instantiated. Look at the comment in the `else`: `v.foo()` there would not be instantiated for `int`. So the code in a discarded branch doesn't have to compile for that type. That's what makes this a replacement for tag dispatch and `enable_if` pairs: with a plain `if`, both branches must compile for every `T`, and `std::to_string` of a `const char*` would fail.

Where does this leave concepts? **`if constexpr` is for branching inside one function. Concepts are for choosing between functions.** You'll use both, and the exercise's solution uses both.

DO: Run `./build/demos/s03/demo_s03_if_constexpr_dispatch`. Expected output: `int 1 | float 2.500000 | other`. Point at `other`: "`describe("x")`: T is `const char*`, not integral, not floating, so the else branch." Point at `2.500000`: "and that's `std::to_string` of a double, six decimals, which is why the exercise uses `std::format` for doubles." Recovery: "Output is int one, float two-point-five, other."

ASK: "What does `describe(true)` print?" (pause) Expect a guess of "other". Say: "`int 1`. `bool` is an integral type. So is `char`. `describe('A')` prints `int 65`. That's the same gotcha you'll hit with `std::integral auto` in the exercise, and it's worth a test."

IF AHEAD: The discarding rule only applies inside a template. In a non-template function, `if constexpr` still picks a branch at compile time, but both branches are fully checked, so you can't use it to hide code that doesn't compile. A related trap: `static_assert(false, "unsupported type")` in an `else` branch used to be ill-formed even when discarded, so people wrote `static_assert(dependent_false<T>)`. C++23 fixed that with P2593, applied as a defect report, and current GCC and Clang accept `static_assert(false)` in a discarded branch of a template. On an older compiler, keep the `dependent_false` trick.

IF BEHIND: "`if constexpr` discards the untaken branch; one function replaces an `enable_if` overload set."

Bridge: you've seen `is_integral_v` twice now. Where did the `_v` come from?

18. Variable templates and the `_v` / `_t` aliases · C++14 · target 0:39, ~2 min

A variable template is a constant parameterized on a type. C++14.

`pi<T>` is pi, converted to T. One constant, every precision. `is_small_v<T>` is your own trait: true if T fits in a pointer.

Now the first `static_assert`: `pi<float> > pi<double>`. That's true. **`float` rounds pi up.** The nearest float to pi is about 3.14159274, which is bigger than the double value 3.14159265. The note on this slide says the compiler taught us that with a failed `static_assert` while this demo was being written. That's the "compile-time unit test" idea catching a wrong assumption, for free.

(pause)

The bottom is the practical part. C++11 wrote `std::is_integral<T>::value` and `typename std::remove_const<T>::type`. Now you write `std::is_integral_v<T>` and `std::remove_const_t<T>`. To be precise about history: **the `_t` aliases came in C++14, and the `_v` variable templates came in C++17.** The slide's code comment lumps both under C++17; the `_t` ones are older.

The practical win of `_t` is that it kills the `typename` keyword in front of dependent types, which was the source of more confusing errors than anything else in C++11 template code.

DO: Run `./build/demos/s03/demo_s03_variable_templates`. Expected output: `3.1415927 3.141592653589793`. Point at the float's last digit, 7: "rounded up." Recovery: "Seven digits of float ending in seven; that's the round-up."

IF AHEAD: One honest portability point about this very file: `!is_small_v<long double>` assumes `long double` is bigger than a pointer. On x86-64 Linux it's 16 bytes, so the assertion holds. On Apple Silicon and on MSVC, `long double` is the same as `double`, 8 bytes, the same size as a pointer, and that `static_assert` fails. This is the right failure: the build caught a platform assumption. In embedded code, the habit is exactly this: `static_assert(sizeof(T) == N)` on every type whose layout crosses a wire or a register boundary, so a new target or a new compiler can't change it silently.

IF BEHIND: "Variable templates are parameterized constants; `_t` is C++14, `_v` is C++17, and both end `::type` and `::value`."

Bridge: Session 2 had a template that knew its own arguments. How?

19. CTAD and deduction guides · C++17 · target 0:41, ~2 min

In Session 2 we wrote `std::visit(overloaded{lambda1, lambda2}, v)`. `overloaded` is a class template. Nobody wrote its template arguments. That's class template argument deduction, CTAD, C++17.

Look at the first line. `overloaded` inherits from every `Fs` and pulls in every `operator()`. In C++17, deduction from `overloaded{l1, l2}` needed the commented-out guide: "when constructed from `Fs...`, deduce `overloaded<Fs...>`". **C++20 deduces aggregates automatically, so the guide is gone.** If you're on C++17, or on an older Clang, you still need that line.

(pause)

A deduction guide is a rule: when constructed from these argument types, deduce these template arguments. Most types never need one; the constructors already say enough.

`Wrapper` shows when you do want one. The constructor takes `const T&`. Without the guide, `Wrapper w{"text"}` would deduce `T` as a character array, which isn't what anyone wants. The guide says: from a `const char*`, deduce `Wrapper<std::string>`. And it wins, because when an implicit guide and a user-written guide match equally well, the non-template guide is preferred.

DO: Run `./build/demos/s03/demo_s03_ctad_guides`. Expected output: `text 3`. Point at `main`: `Wrapper w{"text"}` is a `Wrapper<std::string>`, and the `overloaded` visitor has no guide. Recovery: "Output is text and three; `w.value` is a `std::string`."

IF AHEAD: CTAD has a review hazard: `std::vector v{5, 1}` deduces `vector<int>` with two elements via the initializer-list constructor, not five ones. And `std::vector v2{v}` gives a copy, not a vector of vectors, because the copy deduction candidate is preferred. Some style guides ban CTAD on containers for this reason and allow it on lock guards, pairs, and visitors. Compiler support for aggregate

CTAD: GCC has had it since around GCC 10 or 11; Clang only got it in Clang 17. On anything older, keep the guide.

IF BEHIND: "CTAD deduces class template arguments; C++20 aggregates need no guide; write one only when you want a different type than the argument's."

Bridge: a short one on template parameters that are values.

20. `auto` non-type template parameters · C++17 · target 0:43, ~1 min

`template <auto N>`: a template parameter that's a value, with its type deduced from the argument.

`Constant<42>` has an `int`. `Constant<'x'>` has a `char`. **It replaces the C++11 dance of `template <typename T, T N>`, where you had to say the type and then the value.** `std::integral_constant<int, 42>` is that dance.

(beat)

Preview: in C++20 the value can be a class type, which is how you get a compile-time string as a template argument. That's appendix slide 56, `FixedString`; read it in the PDF.

DO: Optional. `./build/demos/s03/demo_s03_auto_nttp` prints `7 temp_core`. Skip if past 0:53.

IF AHEAD: `template <auto... Vs>` gives you a heterogeneous pack of values, which is how you build a compile-time list of mixed constants, for example a register map with different-width register addresses. And `decltype(N)` inside the template gives you the deduced type when you need it.

IF BEHIND: Skip the slide with "`template <auto N>`: a value parameter whose type is deduced."

Bridge: the C++17 takeaway.

21. C++17 templates takeaway · target 0:44, ~1 min

Fold expressions end recursive variadics. `if constexpr` ends tag dispatch. `_v` and `_t` end `::value` and `::type`. CTAD ends most `make_` helpers. Not all: `make_unique` and `make_shared` still exist for good reasons, because they do the allocation.

Templates stopped being a specialist skill in C++17. **C++20 makes them checkable.**

(pause)

Monday morning: find one recursive variadic template and fold it.

Time check: 0:45, and lab 3.2 starts now. Block three starts at 1:00 on the clock; the five minutes between the lab and it are the buffer for block one, not extra lab time. If you have them, take a break.

IF BEHIND: Read the bold lines. Start the lab on time.

After the lab: back to that error message.

22. Lab 3.2: C++17 templates · target 0:45, ~10 min

Ten minutes, three TODOs. A fold, an `if constexpr`, a deduction guide. The output must not change: the program prints the same four lines before and after.

> DO: Lab slide up, starter link in chat. One quiet minute. Then: "Every TODO has the answer's shape in the comment above it. The thinking is in the second half of each one."

What to watch for:

- TODO 1: `sum` is given in the comment. `count_true` is the one to think about. A unary fold over + works on `bool`s, because they promote to `int`, and it prints 3; but call it with no arguments and GCC says "fold of empty expansion over operator+". The binary form, `(0 + ... + (bs ? 1 : 0))`, handles the empty pack. That is slide 16's gotcha, in their own file.
- TODO 1: someone will write `(bs ? 1 : 0 + ...)`. The fold needs its own parentheses around the whole expression and around the operand: `((bs ? 1 : 0) + ...)`.
- TODO 2: anyone who writes a plain `if` instead of `if constexpr` gets "no matching function for call to `to_string(const char*&)`" and "`basic_string(int&)`". Both branches were instantiated for every `T`. That error is the lesson: the discarded branch is the feature.
- TODO 2: delete the three tag structs and the three `describe_impl` overloads too. People leave them in and the lab still builds; say the point is one function, not four plus a trait expression.
- TODO 3: `std::pair p{3, 0.25}` just works. `Range r{0, 1.0}` without the guide: "class template argument deduction failed", then "deduced conflicting types for parameter 'T' ('int' and 'double')". The guide in the comment resolves it to `Range<double>`. If someone writes `Range r{0.0, 1.0}` instead, that also compiles, and it is a fair answer: the guide exists so callers do not have to.

At seven minutes: solution link. Two minutes: `sum` and `count_true` side by side with the recursive pairs, the one `describe`, and the guide line under `Range`.

> ASK: "Which of the three could you have written in C++14?" Expect: "none". Then: "Right. Fold expressions, `if constexpr`, and CTAD all landed in C++17 together, and that is the year templates stopped being a specialist skill."

At ten minutes, move on. Block 3 starts at 1:00; if you are early, the five minutes before it are the buffer for the first half, not extra lab time.

---

23. The problem concepts solve · demo · target 1:00, ~3 min

Compiler Explorer, two panes, same call: `serialize(ParseError::EmptyLine)`.

> DO: Switch to Compiler Explorer tab 2. Left pane: the starter's overloads. Right pane: the solution's constrained templates. Both GCC 14. Let both compile. On the left, scroll

the candidate list; don't read it all. On the right, put the cursor on the line that says `'telemetry::ParseError' does not satisfy 'StringLike'` (or whichever concept is listed for each candidate). Recovery if Compiler Explorer is slow or down: stay on the slide, which shows the right-hand output verbatim, and say "this is the output, captured from GCC 14."

Left: the starter. Eight candidates, and for each one, a note about what didn't convert. You know this one.

Right: the solution. Each candidate now says `constraints not satisfied`, and then names the concept: `ParseError` does not satisfy `StringLike`. **The error names the requirement. That is the feature.**

(pause)

Let that land. The compiler isn't telling you "I tried this and the types didn't line up". It's telling you "this function requires something to be string-like, and your type isn't". That's the language the library author was thinking in, finally showing up in the diagnostic.

This is exercise task eight, at home. It's the most persuasive thirty seconds in the session, so do it yourself on your own code.

> ASK: "If you got the right-hand error at 4 p.m. on a Friday, how long would it take to fix?" Expect: "a minute" or "seconds". Say: "And the left-hand one? That's the difference. The fix didn't change. The time to find it did."
>
> IF BEHIND: Skip the live demo; read the slide's right-hand error and the bold line.

Bridge: so what is a concept, exactly?

24. A concept is a named predicate on types · C++20 · target 1:03, ~1 min

A concept is a compile-time boolean, parameterized on a type, with a name you can use in a signature. That's all.

`StringLike` is: `T` is convertible to `std::string_view`. It uses a standard concept, `std::convertible_to`. Then three `static_asserts`: `std::string` is string-like, `const char*` is string-like, `int` isn't.

(beat)

That's the same thing as a `constexpr bool` variable template, from three slides ago, with two differences: it has a place in the grammar, so you can write its name where a type would go, and the compiler understands its structure, so it can compare concepts against each other. That second part is subsumption, and it's coming.

In the exercise: `StringLike`, `SerializableRange`, and `Serializable`. Tasks three and four.

> IF AHEAD: Because a concept is just a boolean, `static_assert(Concept<T>)` is a free unit test of your type's interface. Writing `static_assert(std::regular<SensorConfig>)` next to a struct's definition

documents and enforces that it's copyable, default-constructible, and equality-comparable, and the build breaks the day somebody deletes the copy constructor.

IF BEHIND: "A concept is a named compile-time boolean on types."

Bridge: there are four ways to use one.

25. Four ways to constrain · C++20 · target 1:04, ~2 min

Same constraint, four spellings. All four functions on the slide are identical in behavior.

One: a `requires` clause after the template head. `template <typename T> requires StringLike<T>`.

Two: a trailing `requires` clause after the parameter list.

Three: the concept name in place of `typename`. `template <StringLike T>`.

Four: abbreviated. `const StringLike auto& s`. No template line at all.

(pause)

When to use which. **Use four, the abbreviated form, when it's one concept on one parameter. Use three when you need to name the type in the body. Use one when the constraint involves several parameters or is a compound expression.** Two, the trailing form, you'll almost never write, except on a member function of a class template that isn't itself a template: `void sort() requires std::totally_ordered<T>;` There's no template head to hang it on, so it goes at the end.

DO: Run `./build/demos/s03/demo_s03_concepts_basics`. Expected output: `3 2 1 3 3`. Point at `len2("ab")` giving 2: "a character array is convertible to `string_view`, and the view stops at the null." Recovery: "Three, two, one, three, three."

IF AHEAD: The form-one gotcha. In a `requires` clause, you can't write `requires !StringLike<T>`. The grammar only allows primary expressions there, so negation needs parentheses: `requires (!StringLike<T>)`. GCC tells you "expression must be enclosed in parentheses". Inside a concept definition or a nested requirement, the parentheses aren't required. People hit this the first time they write a negative constraint, which in the exercise is task four.

IF BEHIND: "Four spellings; use the abbreviated form for one concept on one parameter."

Bridge: the abbreviated form, side by side with what it replaced.

26. Abbreviated function templates · C++20 · target 1:06, ~1 min

Left: the same constraint in `enable_if` form. The comment says "read it aloud to a colleague". Try it: "template, typename T, typename std enable if, std is convertible, T, string view, value, int, type, equals zero". (beat) Nobody can review that.

Right: `std::size_t len(const StringLike auto& s)`. **`Concept auto` in a parameter list is a template parameter with a constraint, and there's no template line.**

And from the exercise: `serialize(std::integral auto v)` and `serialize(std::floating_point auto v)`. Those two lines replace four integer and floating-point overloads in the starter. That's task three.

(pause)

Note the left side is labeled C++11 but uses `std::string_view`, which is C++17. It's the C++11 *technique*.

> IF AHEAD: `std::integral auto` accepts `bool`, `char`, `char8_t`, and so on. If `serialize(true)` should print `true` and not `1`, you need a more specific overload for `bool`, and the more-specific rule is the next-but-two slide. Also: an abbreviated function template is a template, so it lives in a header, and each distinct argument type is a separate instantiation, which is the same code-size story as any other template.
>
> IF BEHIND: "`Concept auto` is a constrained template parameter with no template line."

Bridge: how do you write your own concept when no standard one fits?

27. `requires` expressions · C++20 · target 1:07, ~2 min

A `requires` expression defines a concept by what must compile. There are four kinds of requirement, and this slide shows all four.

Look at the `Range` concept. `t.begin();` and `t.end();` are simple requirements: this expression must compile. `typename T::value_type;` is a type requirement: this type must exist. `{ t.size() } -> std::convertible_to<std::size_t>;` is a compound requirement: the expression compiles and its type satisfies a concept. The next line adds `noexcept`: `t.empty()` compiles, doesn't throw, and returns exactly `bool`. And the last one, `requires !std::same_as<T, std::string>;`, is a nested requirement: another constraint must hold.

(pause)

The expressions inside a `requires` expression are never evaluated. They're checked for whether they would compile. So `t.begin()` doesn't call anything; it asks the compiler "is this well-formed for this T?"

Bottom: `vector<int>` is a `Range`. `std::string` isn't, because the nested requirement excludes it. `int` isn't, because there's no `int::value_type`.

The exercise's `SerializableRange` uses simple requirements for `begin` and `end`, plus `!StringLike` as the exclusion. You'll see why the exclusion is needed two slides from now.

> IF AHEAD: The parameter list of a `requires` expression, `requires(T& t)`, just introduces names to use in the expressions. It doesn't create objects. So

`requires(T& t)` versus `requires(const T& t)` matters a lot: with `const T&`, `t.begin()` checks the const overload. A common bug is writing `requires(T t)` and then being surprised that a non-copyable type passes or fails based on things that have nothing to do with the requirement. There's also an ad hoc form in the demo file, `requires requires(T t) { t.size(); }`, which is legal and looks like a typo. Prefer a named concept: it subsumes, it documents, and the error message uses its name.

IF BEHIND: "Simple, type, compound, nested: four kinds of requirement; the expressions are checked, never run."

Bridge: before writing your own, check the standard library.

28. The standard concepts library · C++20 · target 1:09, ~1 min

The table is a reference. Don't read it; skim the rows. Core language concepts like `same_as`, `convertible_to`, `integral`, `floating_point`. Object concepts: `copyable`, `movable`, `semiregular`, `regular`. Comparison concepts. Callable concepts: `invocable`, `predicate`. Iterator and range concepts, which are Session 4.

(beat)

Prefer these over your own. They subsume each other correctly, and readers already know their names.

The important semantic one is `regular`: copyable, default-initializable, and equality-comparable. It also promises that a copy compares equal to the original. The compiler can't check that promise. It's documentation the compiler holds you to only in syntax.

One header correction for the table: `three_way_comparable` lives in `<compare>`, not `<concepts>`.

IF AHEAD: Two library concepts worth knowing by name for embedded work: `std::invocable<F, Args...>` for callbacks, which replaces "takes a `std::function`" with "takes anything callable with these arguments" and no type erasure; and `std::predicate` for filters. Combining them with `std::regular_invocable` documents that the callback has no side effects that matter, which is a promise, not a check.

IF BEHIND: "Prefer standard concepts; they subsume correctly and people know them."

Bridge: subsumption. Slow down.

29. Subsumption: the more constrained overload wins · C++20 · target 1:10, ~3 min (slow down)

When two constrained overloads both match a call, the compiler needs a rule to pick one. That rule is subsumption.

Look at the code. `HasSize` requires `t.size()`. `Container` is defined as `HasSize<T> && requires { t.begin(); t.end(); }`. Read that out loud: a container is something that has a size, and also has begin and end.

Two overloads of `describe`. One takes `const HasSize auto&`. One takes `const Container auto&`.

(pause)

Call `describe(std::string{})`. A `std::string` has `size()`, so it's `HasSize`. It also has `begin()` and `end()`, so it's a `Container`. Both overloads match. Ambiguous?

(pause)

No. **The compiler picks `Container`, because `Container` is `HasSize` and more.** The compiler breaks each concept into its pieces, called atomic constraints, sees that `Container`'s pieces include all of `HasSize`'s pieces, and concludes `Container` is more constrained. That's subsumption: when both match, the one whose constraints include the other's wins.

And it can only see that because `Container` is written as a conjunction that names `HasSize`. Hold that thought; the next slide is what happens when you don't.

(beat)

Now connect it to the exercise. The notes say the exercise's `SerializableRange` is `!StringLike<T> && requires { begin, end }`. Why the `!StringLike`? Because a `std::string` is string-like, so the `StringLike` overload matches. And a `std::string` has `begin` and `end`, so a range overload would match too. Neither concept includes the other. **Without the exclusion, `serialize(std::string{})` is ambiguous.** The `!StringLike` takes `std::string` out of the range overload entirely. That's exclusion, not subsumption: negation is an atomic constraint the compiler doesn't look inside.

> DO: Run `./build/demos/s03/demo_s03_subsumption`. Expected output: `container has size`. Point at `describe(std::string{})` giving `container` and `describe(Sized{})` giving `has size`: `Sized` has only `size()`, so only `HasSize` matches. Recovery: "Container for the string, has-size for the struct with only size."
>
> ASK: "What does `describe(std::vector<int>{})` print?" Expect: "container". Say: "Yes. And what about a C array, `int a[3]`?" (pause) Expect hesitation. Say: "Neither. A C array has no member `size()`, so it's not even `HasSize`. No viable function. The standard library uses `std::ranges::size`, which works on arrays, exactly to avoid that. Session 4."
>
> IF AHEAD: Subsumption also beats unconstrained overloads: a constrained template is preferred over an unconstrained one with the same signature. That's the standard way to write a fast path: a generic `serialize(const T&)` and a constrained `serialize(const T&) requires std::is_trivially_copyable_v<T>`. Note the README for exercise task four says that without `!StringLike` a `std::string` "would serialize as a list of chars". With the starter's design, the more

accurate statement is that the call is ambiguous and doesn't compile. Either way, the exclusion is required.

IF BEHIND: "When both overloads match, the one whose concept includes the other's wins; that's why `SerializableRange` excludes `StringLike`."

Bridge: here's the trap.

30. Subsumption only sees named concepts · C++20 · target 1:13, ~2 min (slow down)

`Container2` is the same idea written differently: one `requires` block with `size`, `begin`, and `end`. Logically, it's exactly `HasSize` and more. Two overloads, `describe2(HasSize)` and `describe2(Container2)`.

Call `describe2(std::string{})`. (pause) Ambiguous. Compile error.

Why? **Two `requires` expressions are never compared for subsumption, even if they're textually identical.** The compiler doesn't look inside a `requires` block and reason about it. To the compiler, `Container2` is one opaque atomic constraint. `HasSize` is a different opaque atomic constraint, written in a different place. Neither includes the other, so neither is more constrained.

(pause)

This is deliberate. Comparing arbitrary expressions for logical implication is undecidable in general, so the standard says atomic constraints are only identical when they come from the same expression in the same place in the source. In practice, that means from the same named concept.

The rule: build concepts as conjunctions of named concepts. Ad hoc `requires` blocks are leaves, not layers.

That's the one subsumption rule that bites in practice, and the fix is always the same: name the pieces.

DO: Optional. Uncomment line 30 in `subsumption.cpp`, `describe2(std::string{})` (move it into `main`), and rebuild the target. Expect "call of overloaded 'describe2(std::string)' is ambiguous" and two candidates listed. Revert. Recovery: "The comment on line 30 is the error."

IF AHEAD: The same rule means copying a standard concept's definition into your own code breaks subsumption. If you write your own `my_integral` as `std::is_integral_v<T>`, it doesn't subsume or get subsumed by `std::integral`, even though they're the same predicate. So two overloads, one on `std::integral` and one on `my_integral && something`, can be ambiguous. Another reason to build on standard concepts by name rather than reimplementing them.

IF BEHIND: "Subsumption only works through named concepts; inline `requires` blocks are opaque."

Bridge: concepts can also ask questions about your own code.

31. Concepts as questions about your API · C++20 · target 1:15, ~2 min

This is exercise task four, hand-typed from the solution.

`Serializable` is defined as: given a `const T& t`, the expression `serialize(t)` compiles and returns exactly `std::string`. Read the comment: "some `serialize` overload accepts a `T`".

(pause)

Now look at the `static_asserts`. `Serializable<int>`: yes, the integral overload takes it. `Serializable<std::vector<std::vector<int>>>`: yes, because the range overload accepts a vector whose elements are serializable, and those elements are vectors whose elements are ints. Recursion, checked by the compiler. And `!Serializable<ParseError>`: no overload accepts it.

That last line is documentation and enforcement in one: a question about your API, answered by the compiler, on every build. If somebody later adds a `serialize(ParseError)` overload by accident, or a conversion that lets it through, this line fails.

This isn't constraining a template. It's asking. You can use the same concept in `if constexpr (Serializable<T>)` to pick a fallback, as a constraint on other templates, and as the name that shows up in an error.

> ASK: "Where in your codebase would a line like `static_assert(!Serializable<X>)` have caught something?" Expect: messages that should never go on the wire, types that should never be logged, secrets that should never be formatted. Say: "Exactly. A `static_assert` that a key type isn't formattable is a cheap security control."
>
> IF AHEAD: The name-lookup gotcha. A concept that calls `serialize(t)` sees the `serialize` overloads declared before the concept, plus whatever argument-dependent lookup finds at the point of use. For a type in your namespace, ADL finds your overloads. For `int`, there's no associated namespace, so only overloads declared before the concept count. If someone moves the integral overload below the concept definition, `Serializable<int>` silently becomes false. Keep the concept after all the overloads it asks about, and keep a `static_assert` for at least one fundamental type, which is exactly what the solution does.
>
> IF BEHIND: "A concept can ask whether your own API accepts a type; `static_assert(!Serializable<ParseError>)` documents and enforces it."

Bridge: some opinions on design.

32. Concept design guidance · target 1:17, ~1 min

This slide is opinionated, and I'll say so.

Name the capability, not the type. `Serializable`, `Hashable`, `Range`. Not `IsVector`. A concept named after a type is just a type check with extra steps.

Prefer standard concepts where one fits. We've said why.

(pause)

The third bullet is the one people argue about: constrain with the weakest concept that makes the body compile. Over-constraining rejects valid callers. Under-constraining gives bad errors, not wrong code: the template still fails to compile, just deeper inside. The C++ Core Guidelines push in a slightly different direction, toward concepts with meaningful semantics, rule T.20, rather than minimal syntactic ones like "has `operator+`". **So the honest version is: the weakest meaningful concept, preferably a standard one, that makes the body compile.**

Semantic requirements exist. `std::regular` promises that copies compare equal. `totally_ordered` promises transitivity. The compiler checks syntax only. Document the rest.

Don't constrain for its own sake. A helper template used with two types in one `.cpp` file doesn't need a concept. A library boundary does.

> ASK: "Where's the library boundary in the telemetry program?" Expect: `serialize`, the parser interface, `SensorStats`. Say: "`serialize` is the one that's called from everywhere, so that's where we spent the effort."
>
> IF AHEAD: A code-review heuristic: if a template has a comment that says "T must be...", that comment is a concept waiting to be written. The syllabus puts it the same way: concepts wherever a template previously had a comment explaining its requirements. The comment can go stale; the concept can't.
>
> IF BEHIND: "Name the capability, prefer standard concepts, and constrain at library boundaries."
>
> IF AHEAD: Two C++20 features that pair with concepts are in the appendix. Slide 55: template lambdas, `[]<typename T>(std::span<const T>)`, and lambdas in unevaluated contexts, which give you a one-line RAII deleter: `std::unique_ptr<FILE, decltype([](FILE* f) { if (f) std::fclose(f); })>`. Slide 56: class types as template arguments, the `FixedString` idiom, and why the exercise's `SensorConfig` cannot be one (a `string_view` member is not structural).

Bridge: back to error messages, one more time, side by side.

33. Concepts vs SFINAE: the error messages · C++20 · target 1:18, ~1 min

Same constraint, same wrong call, `twice(2.5)`, GCC 14.

Left: `enable_if`. No matching function. Candidate: a template with an anonymous `enable_if` parameter. Substitution failed. "No type named `type` in `struct std::enable_if<false, int>`."

Right: concept. No matching function. Constraints not satisfied. **`double` does not satisfy `integral`.**

(beat)

The right side says what was required and what failed to meet it. The left side says something about a struct you never wrote.

> DO: Optional. Compiler Explorer tab 3, `concepts_vs_sfinae.cpp` with `-DSHOW_ERRORS`, GCC 14 and Clang 18 panes. Point at the Clang pane's "because 'double' does not satisfy 'integral'". Recovery: stay on the slide.
>
> IF AHEAD: When a concept is built from other concepts, GCC by default only shows the top level of the failure. `-fconcepts-diagnostics-depth=2` (or 3) makes it expand into the nested concept that actually failed. Worth adding to a debug build preset.
>
> IF BEHIND: Point at the right column: "`double` does not satisfy `integral`. That's the whole argument."

Bridge: the takeaway.

34. Concepts takeaway · target 1:19, ~1 min

Constrain every template parameter at a library boundary with the weakest meaningful concept that makes the body compile. Build concepts from named concepts so they subsume.

The error message is the feature. Everything else concepts do, `enable_if` could do, worse.

(pause)

Monday morning: replace one `enable_if` with a concept, and compare the two error messages.

Time check: 1:20, and lab 3.3 starts now. Its TODO 2 is exactly that comparison, on a file small enough to read both errors in full.

> IF BEHIND: Read the bold lines. Start the lab on time.

After the lab: the last feature. C++23, and it fixes three things at once.

35. Lab 3.3: concepts · target 1:20, ~10 min

Ten minutes, three TODOs. Name a requirement, put it on a template, then add a second overload that only a range satisfies. TODO 2 is the one with the error message: do it twice, once before the constraint and once after.

> DO: Lab slide up, starter link in chat. One quiet minute. Then: "Before you write anything, uncomment the `to_json(Opaque{1})` line in `main`, build, and look at where the error is. Then comment it back and start TODO 1."

What to watch for:

- TODO 1: the concept is in the comment. The stumble is placing it above the `serialize` overloads; then `Serializable<int>` is false and the first `static_assert` fails. The concept asks about the overloads declared before it (slide 31's name-lookup note).

- TODO 2, before the constraint: "no matching function for call to `serialize(const Opaque&)`", reported inside `to_json`, with "required from here" pointing back at `main` and three candidate notes. After: "no matching function for call to `to_json(Opaque)`", then "constraints not satisfied" and "the required expression `serialize(t)` is invalid", on the call site. **Same bug, same fix; the error moved from inside the template to the line that made the mistake, and it names the requirement.** Make everyone read both.
- TODO 2: any of the four spellings works. The solution uses `template <Serializable T>`. Someone who writes `requires Serializable<T>` after the parameter list is also right.
- TODO 3: the concept is in the comment; `#include <ranges>` is in the hint. GCC 14 happens to compile without the include through `<vector>`; libc++ may not. The loop needs a separator that starts empty and becomes `","` after the first element, or the output has a leading or trailing comma and does not match.
- TODO 3: nobody should hit an ambiguity, and someone will ask why not. `std::vector<Reading>` is not `Serializable` (no `serialize` overload takes it), so only the range overload is viable. Point at the comment in the starter: if a type satisfied both, the overload whose constraint subsumes the other would win, and that only works because `SerializableRange` is written with named concepts.

At seven minutes: solution link. Two minutes: the concept, the one-word constraint on `to_json`, the range overload. Then the two error messages one more time if anyone did not run the before version.

> ASK: "Where did the error move to?" Expect: "to the call", "to main". Then: "From three levels down inside a template to the line that made the mistake, and it says what was required. That is what concepts are for; everything else they do, `enable_if` could do worse."

At ten minutes, move on. Block 4 is short and ends with the last lab at 1:45.

---

36. Three problems, one feature · C++23 · target 1:30, ~2 min

Three problems you've probably all written around.

One: `const` duplication. Every accessor written twice, `const` and non-`const`, with identical bodies. Sometimes four times, if you also care about rvalues.

Two: CRTP. A base class that needs its derived type as a template parameter so it can call down into it.

Three: recursive lambdas. A lambda can't name itself, so recursion needed `std::function` or a Y-combinator.

(pause)

All three are the same problem: a member function doesn't know the type or value category of the object it was called on. Inside a member function, `this` is a pointer to the

class you're in, with the `const` you wrote, and nothing about whether the object was a temporary.

C++23 lets you declare that object as an explicit parameter and deduce it. The paper is P0847, "deducing `this`". **It's the most consequential C++23 language feature for class design.**

> ASK: "Show of hands: who has a class with a `const` and a non-`const` version of the same accessor?" Expect nearly everyone. Say: "And who has had them drift apart, where someone fixed a bug in one and not the other?" (pause) "That's the bug this feature deletes."
>
> IF BEHIND: "`const` duplication, CRTP, and recursive lambdas are one problem: the member function doesn't know its object's type; deducing `this` fixes it."

Bridge: here's what it looks like.

37. Explicit object parameter · C++23 · target 1:32, ~3 min (slow down)

This is exercise task seven.

Left: `SensorStats::add` the classic way. It updates the stats and returns `*this` by reference, so you can chain: `stats.add(...).add(...)`. That's fine on an lvalue. On a temporary, `SensorStats{}.add(...).add(...)` returns an lvalue reference to the temporary. If you copy the result out, it copies, because it looks like an lvalue. If you bind a reference to it, it dangles at the semicolon.

(pause)

Right: C++23. Look at the signature. `template <typename Self> constexpr Self&& add(this Self&& self, double v, Status s)`. The first parameter has the keyword `this` in front. That's the explicit object parameter. It's the object the function was called on.

`Self&&` with a deduced `Self` is a forwarding reference, the same as in any perfect-forwarding template. So: **call it on an lvalue `stats`, and `Self` is `SensorStats&`; call it on a temporary, and `Self` is `SensorStats`, so `self` is an rvalue reference.**

(pause)

The return type is `Self&&`, and the body returns `std::forward<Self>(self)`. So the return type follows the object's value category. On an lvalue, it returns `SensorStats&`. On a temporary, it returns `SensorStats&&`, which means `auto s = SensorStats{}.add(1, ok).add(2, ok);` moves through the chain instead of copying. **One definition serves both.** And it's `constexpr`, so in the exercise you'll evaluate a three-sample accumulator inside a `static_assert`.

Inside the body there is no `this` pointer. You write `self.count`, `self.sum`. Forgetting that is the first compile error everyone hits.

> ASK: "What would you have written in C++11 to get the same behavior?" (pause)
> Expect: "two overloads with & and && ref-qualifiers". Say: "Right, `add(...) &`

returning `SensorStats&` and `add(...) &&` returning `SensorStats&&`. Two bodies, or one body and a cast. Add `const` and it's four."

IF AHEAD: Three restrictions to know before you use this in review. An explicit-object member function can't be `virtual`, can't be `static`, and can't have `const` or `&` qualifiers after the parameter list, because the object parameter already says all that. Second, it's a template, so it moves into the header, and if it used to be defined out-of-line in a `.cpp`, that's a change to your exported symbols. Third, the lifetime trap doesn't go away: `auto&& r = SensorStats{}.add(1, ok);` binds `r` to a member of a temporary that dies at the semicolon. Returning `Self&&` doesn't extend anyone's lifetime. If you want chaining on temporaries to be safe to bind, return `Self` by value in the rvalue case instead.

IF BEHIND: "`this Self&& self` makes the object a forwarding-reference parameter; one `add` returns `&` on lvalues and `&&` on temporaries."

Bridge: the `const` duplication case.

38. Deduplicating `const` overloads · C++23 · target 1:35, ~2 min

`Config` holds a vector of names. The commented-out C++11 code is the familiar pair: a `const at` returning `const std::string&`, and a non-`const at` returning `std::string&`. Same body twice.

C++23: one function. `template <typename Self> auto&& at(this Self&& self, std::size_t i)`. **`Self` deduces as `Config&`, `const Config&`, or `Config&&`, and the return type follows.**

(pause)

The body is `return std::forward_like<Self>(self.names[i]);` Look at that function: `forward_like`. It's C++23, and it's the companion utility. `std::forward<Self>` would forward `self`. But we're returning a member, `self.names[i]`, and we want the member to have the same constness and value category as the object. **`forward_like<Self>(member)` gives the member the object's category: a `const&` object gives a `const&` member, an rvalue object gives an rvalue member.**

So the four-overload problem, `const` and non-`const` times lvalue and rvalue, collapses to one.

DO: Run `./build/demos/s03/demo_s03_deducing_this`. Expected output, four lines: `RPM x`, `circle`, `square`, `55 ab`. Point at `main` line 54: `c.at(0) = "RPM"` assigns through a non-`const Config&`, so `at` returned `std::string&`. Line 55: `cc.at(0)` through a `const Config&` returned `const std::string&`, and `Config{{"x"}}.at(0)` on a temporary returned `std::string&&`. Recovery: "The first line, `RPM x`, proves the non-`const` path wrote and the `const` path read."

ASK: "What would happen if the body used `std::forward<Self>(self).names[i]` instead of `forward_like`?" Expect uncertainty. Say: "For lvalues, the same thing. For a temporary, member access on an rvalue gives an rvalue, so you'd also get `&&`. The difference shows up with containers: `names[i]` calls `vector::operator[]`, which returns `std::string&` whether or

not the vector is an rvalue. `forward_like` fixes that by applying the category explicitly. That's why it exists."

IF AHEAD: Returning `auto&&` from a function that can be called on a temporary is the same lifetime trap as the last slide. `const auto& name = Config{{"x"}}.at(0);` dangles. Range-for over a member of a temporary is the classic version of this bug; C++23's P2718 extends temporaries in range-for initializers, but only GCC 15 and Clang 19 have it, per the toolchain matrix. On the course baseline, it still dangles.

IF BEHIND: "One `at` instead of two; `forward_like` gives the member the object's constness and category."

Bridge: CRTP.

39. CRTP without the template parameter · C++23 · target 1:37, ~2 min

The commented code is classic CRTP. `template <typename Derived> struct Shape11`, and `draw()` does `static_cast<Derived*>(this)->draw_impl()`. `Circle` inherits from `Shape11<Circle>`, naming itself. It works, it's fast, and it's confusing the first time anyone sees it. It's also easy to get wrong: if `Square` accidentally inherits from `Shape11<Circle>`, copy-paste style, the `static_cast` is undefined behavior and it compiles cleanly.

(pause)

C++23: `struct Shape { void draw(this auto&& self) { self.draw_impl(); } };` No template parameter on the base. No `static_cast`. `Circle` and `Square` just inherit from `Shape`.

How does `Shape::draw` know about `Circle`? **`this auto&& self` deduces to the type of the object at the call site, and when you call `Circle{}.draw()`, that type is `Circle`, not `Shape`.** The deduction happens where the call is written, so `self` is a `Circle`, and `self.draw_impl()` calls `Circle::draw_impl`. And the copy-paste bug from CRTP can't happen, because there's no type argument to get wrong.

(pause)

One correction to the slide's notes. The notes say the base is a plain struct you can put in a container. Careful. **This is still static polymorphism.** If you hold a `Shape&` and call `draw()`, `self` deduces as `Shape&`, and `Shape` has no `draw_impl`, so it doesn't compile. If you need a heterogeneous container of shapes, you still need virtual functions or a `variant`. What you gain is that the base is a normal class, not a template, so it's easier to name, to forward-declare, and to read.

ASK: "Is there a runtime cost compared to CRTP?" Expect: "no". Say: "None. Both are resolved at compile time and both inline. The difference is purely in what you have to write and what you can get wrong."

IF AHEAD: A subtle one: because `self` is the derived type, `self.member` finds the derived class's member if it hides a base member of the same name. Usually that's what you want. Occasionally it's a surprise, when a derived class accidentally has a

data member with the same name as one in the base. A second one: `draw` is now a template, instantiated once per derived type, which is the same code-size profile as CRTP.

IF BEHIND: "`this auto&& self` deduces the derived type at the call site; CRTP without the template parameter, still static dispatch."

Bridge: and recursive lambdas.

40. Recursive lambdas · C++23 · target 1:39, ~2 min

`fib: [](this auto self, int n) -> int { return n < 2 ? n : self(n - 1) + self(n - 2); }`. The lambda's first parameter is itself. So it can call itself. **A recursive lambda with no `std::function` and no helper.**

Before C++23 you either stored it in a `std::function`, which costs a type-erased indirect call on every recursion and possibly an allocation, or you wrote a Y-combinator, which nobody on your team will review happily.

(pause)

Two details. `this auto self` is by value. The closure has no captures, so it's empty, and copying it costs nothing. And the explicit `-> int` return type is needed: the body calls `self` before the compiler has deduced the return type, so without it you get "use of `auto` before deduction".

Second block, `Builder`: the chaining pattern from slide 37, generalized. Lab 3.4's TODO 3 is this `Builder` with a derived class on top. `Builder{}.add("a").add("b")` moves the temporary through each call. `Builder b; b.add("a")` returns `b` by reference.

The demo prints `55 ab`: `fib(10)` is 55.

IF AHEAD: The by-value `this auto self` also works for lambdas with captures, but then each recursive call copies the captures. For a lambda capturing a big object by value, use `this const auto& self` instead. And remember the usual recursion caveat for embedded code: a recursive lambda is still recursion, with stack depth proportional to the input, which some coding standards prohibit outright.

IF BEHIND: "`this auto self` lets a lambda call itself with no `std::function`."

Bridge: the takeaway.

41. Deducing `this` takeaway · target 1:41, ~2 min

Declare the object as a parameter, `this Self&& self`, and the member function knows its type and value category. One accessor instead of two or four. CRTP without the template parameter. Recursive lambdas.

The by-value trick: `this auto self` for small types. A small type passed by value can travel in registers instead of through a pointer to memory. For something like a `string_view`-sized handle or an iterator-like type, that can be cheaper than `this` by reference. It also means the

function works on a copy, so mutations don't affect the original. That's a feature for a view and a bug for a builder.

(pause)

Monday morning: find one `const`/non-`const` accessor pair and merge it. Check your compiler first: deducing `this` needs GCC 14, Clang 18, or a recent MSVC. If your toolchain is older, this is the one feature today you'll have to wait for.

Time check: leave by 1:43. Two minutes of diagnostics, then lab 3.4 at 1:45.

> IF BEHIND: Read the bold lines. Move on.

Bridge: one live comparison, then the last lab.

42. Reading the diagnostics · demo · target 1:43, ~2 min

Two minutes. This primes the at-home exercise's tasks two and eight.

This repeats the comparison from slide 23, so keep it short. **The thing new here is breaking the sensor table.**

> DO: Compiler Explorer tab 2: flash both panes for ten seconds and say only "starter, nine candidates; solution, the concept by name." Then switch to the editor. In the solution's `config.h` (or your scratch copy), on the sensor you identified before class, set `.min_valid = 500.0` so it is at or above its `max_valid`. Run `cmake --build build`. Expect: "static assertion failed: sensor configuration table is invalid", pointing at the `static_assert(validate(kSensors).empty(), ...)` line. Point at the message and the expression. Revert the edit immediately and rebuild once to confirm green. Recovery if the build is slow or the edit doesn't fail: "The `static_assert` message is what you'll see; try it yourself in task two at home."

(pause)

That's the whole pitch for the exercise's task two. **A bad configuration table didn't make it to a binary.** You wrote a small version of that validator in lab 3.1; at home you write the real one.

Note what you saw: the message and the expression. The specific reason isn't in the error on our compilers. When you write your validator, keep its checks small and obvious, so the reader of a failed build can find the cause in seconds.

> IF BEHIND: Skip the Compiler Explorer reprise; only do the table break.

Bridge: the last lab, and it is the whole of block four in one file.

43. Lab 3.4: deducing this · target 1:45, ~10 min

Ten minutes, three TODOs, one feature. An accessor pair becomes one member, a `std::function` recursion becomes a lambda that names itself, and a builder chains from a derived class without CRTP. Build after each; the first three output lines must not change, and TODO 3 adds a fourth.

DO: Lab slide up, starter link in chat. One quiet minute. Then: "The signature you need is in the comment above each TODO. Inside these functions there is no `this`; everything goes through `self`."

What to watch for:

- TODO 1: the signature is in the comment. The stumble is writing `this->items_` inside it: "'this' is unavailable for explicit object member functions". It is `self.items_`. `std::forward<Self>(self).items_[i]` and plain `self.items_[i]` both pass this lab; the forward matters when a temporary `Buffer` should hand out an rvalue, which `main` does not test. Mention it, do not block on it.
- TODO 1: `b.at(1) = 25` still compiles after the change, and `cb.at(2)` still cannot be assigned to. `Self` deduced as `Buffer&` for the first and `const Buffer&` for the second; the return type followed. One body, both behaviors.
- TODO 2: dropping the `-> int` gives "use of `<lambda>` before deduction of 'auto'". The body calls `self` before the return type is known, so it has to be stated. Also: someone will keep the `std::function<int(int)> fib =` on the left and only change the lambda. It compiles and prints 55, and the type erasure is still there. The fix is `auto fib =`.
- TODO 3: the chaining trick is `Self&&` as the return type plus `return std::forward<Self>(self);`. People write `Self&` and return `self`; `fb.id(7)` still works, but the chained call on the temporary fails with "cannot bind non-const lvalue reference of type `FrameBuilder&` to an rvalue of type `FrameBuilder`".
- TODO 3: before anyone changes anything, the uncommented chain fails with "'class Builder' has no member named 'build'": after `id(8)` the object is a `Builder`. That message is the problem statement; after the change, `self` is a `FrameBuilder&&` at every step and `build()` is found.
- TODO 3, if asked about lifetime: `FrameBuilder{}` is a temporary, and it lives to the end of the full expression, so chaining through it and calling `build()` is safe. Binding `auto&& r = FrameBuilder{}.id(8);` would not be; `Self&&` does not extend anyone's lifetime. That is slide 37's IF AHEAD.

At seven minutes: solution link. Two minutes: the one `at`, the lambda with `self`, and the two setters. Point at the last line of `main`: four calls on a temporary, no copy, no `static_cast`, no template parameter on `Builder`.

ASK: "In `FrameBuilder{}.id(8).payload("pong").build()`, what type does `self` have inside `id` and inside `payload`?" Expect: "FrameBuilder", "an rvalue reference". Then: "`FrameBuilder&&` both times. The base class never names the derived type and never casts; the call site tells it. That is CRTP with the template parameter deleted."

At ten minutes, move on: one minute for the at-home exercise, then the takeaway.

44. Exercise: compile time and concepts · target 1:55, ~1 min to launch

One minute. Open `exercises/s03-compile-time/README.md` at home.

Tasks one to three are today's labs applied to the real program: the CRC table at compile time with a `static_assert` on `0x29B1`, a `consteval` validator for the sensor table, and constrained `serialize`. Then tasks four to eight. **Most of the new tests are `static_asserts`: if it compiles, they passed. And `s03_report_identical` still diffs your report against the starter's, byte for byte.**

(beat)

Task one has a snag in it on purpose: `std::as_bytes`. Appendix slide 49 says why. No toolchain at home? The README has Compiler Explorer links for the whole program, starter and solution. The solution is next week's starter, so finishing is optional, but reading it is not.

> DO: Paste into chat: `exercises/s03-compile-time/README.md` and `cmake --build build && ctest --test-dir build -R s03 --output-on-failure`.

45. Session 3 takeaway · target 1:56, ~2 min

Three functions ran at startup or per call, and now they run at build time. One class of bug, a bad configuration table, **cannot reach a running program.** The generic code got shorter, and its error messages got readable. And the report output didn't change by a single byte.

(pause)

`constexpr` by default, `consteval` when it must, concepts at every library boundary, deducing `this` for every accessor pair.

Point at the timeline handout: the `constexpr` row is the longest on it. That's the story of the first half of today in one row.

Next session: ranges. The `SerializableRange` concept you wrote becomes `std::ranges::input_range`, and the report loop becomes a pipeline. Do the exercise before then; the solution is the Session 4 starter, so if you don't finish, you start from ours.

(beat)

Thanks. Questions in the chat, or stay on for five minutes.

Appendix slides (in the PDF, not presented)

These sections are kept for the `IF AHEAD` moments and for answering questions; the slides sit after the Further reading divider (deck slide 46).

A1. The two halves of today (deck slide 47)

Making the compiler compute. `constexpr` means a function *may* run at compile time. `consteval` means it *must*. `constinit` means a variable is initialized at compile time but stays mutable. The result is tables, checks, hashes, and configuration that cost nothing at runtime, and `static_assert` as a unit test that runs every time you build.

Making the compiler check. That's concepts. A template's requirements go in its signature, where the compiler can check them at the call site, and the error message names the requirement that failed.

(beat)

In between is a bridge. C++17 made templates readable with fold expressions and `if constexpr`. C++20 made them checkable with concepts. And C++23's deducing `this` removes the last big pile of boilerplate in class design.

Keep "may, must, initialized" in your head for the first half. That's the whole vocabulary.

Bridge: let's start where `constexpr` started, which was not very impressive.

A2. What a constant expression refuses · C++11 (deck slide 48)

Undefined behavior is not allowed in a constant expression. That's been true since C++11, and it's the most underrated property of `constexpr`.

Look at the code. `overflow(x)` returns `x + 1`. `at(a, i)` returns `a[i]`. Ordinary functions. Inside `SHOW_ERRORS`, three `static_asserts`. `overflow(INT_MAX)`: signed overflow. The compiler refuses: overflow in constant expression. `at({1, 2, 3}, 5)`: out of bounds. Refused. `uninit()`: reads an uninitialized `int`. Refused.

(pause)

The compiler is a sanitizer for every `constexpr` call it evaluates, and it costs nothing at runtime. Signed overflow, out-of-bounds access, uninitialized reads, use after lifetime ends, null dereference: all of those are hard errors during constant evaluation. Not warnings. Not "maybe the optimizer does something weird". Errors.

So the advice is: write your lookup tables and protocol constants `constexpr`, add a `static_assert` or two, and you get UBSan and ASan for that code at build time, for free, on every build.

> DO: In the terminal, run `g++-14 -std=c++23 -fsyntax-only -DSHOW_ERRORS demos/s03/constexpr_limits.cpp`. Expect three or more errors: "overflow in constant expression", "array subscript value '5' is outside the bounds", and an uninitialized-read error, plus one from the `reinterpret_cast` on the next slide. Point at each message in turn. Recovery: "If the output scrolls past, the three lines to find are overflow, out of bounds, uninitialized. The comments in the file name them."
>
> IF AHEAD: The uninitialized case has a version wrinkle. Before C++20, a `constexpr` function couldn't even declare an uninitialized local. C++20 allows the declaration, but reading it during constant evaluation is still an error. The comment in the file notes Clang rejects the read even earlier, with its uninitialized-variable diagnostic. Also, this is only for what the compiler actually evaluates. A `constexpr` function called at runtime with `INT_MAX` overflows exactly like any other function. The sanitizer runs on the compile-time calls, which is why the `static_asserts` with edge cases are worth writing.

IF BEHIND: "UB in a constant expression is a compile error; the compiler is your sanitizer."

Bridge: there's one more thing it refuses, and you'll hit it in task one.

A3. Also refused: `reinterpret_cast` · C++11 (deck slide 49)

`reinterpret_cast` is never allowed in a constant expression. Never. Not in C++11, not in C++26.

Look at `first_two_bytes`. Under `SHOW_ERRORS`, it takes `s.data()`, reinterprets it as a pointer to `uint16_t`, and dereferences. That's the "read two bytes as a word" idiom you've seen in every protocol parser. In a constant expression, it's an error. The `#else` branch does it properly: two `static_cast`s to `unsigned char`, a shift, an OR.

(pause)

Why? **A constant expression has no memory to reinterpret, only values.** The compiler's evaluator tracks objects and their types. A `uint16_t` view of two `char`s isn't an object that exists, so there's nothing to read.

This is why `std::as_bytes` isn't `constexpr`, and it's the snag you'll hit in exercise task one. The starter's `crc16` takes `std::as_bytes(...)` of the input. When you make `crc16` `constexpr` and add the `static_assert`, the compiler will tell you `as_bytes` isn't usable in a constant expression. The fix in the solution is a template over the byte type with `static_cast<unsigned char>` per byte. I'll let you find that yourselves.

The rest of the list at the bottom: `goto` and inline `asm` can't be *executed* during constant evaluation. Calling a non-`constexpr` function is refused. `std::to_string` isn't `constexpr`, so for integers use `std::to_chars`, which is `constexpr` for integral types since C++23.

> ASK: "If you need to turn bytes into a `uint16_t` at compile time and you don't want to write shifts, what's the C++20 tool?" Expect: `std::bit_cast`. Say: "Right, and `bit_cast` is `constexpr` as long as neither type contains pointers, references, or unions. It copies the object representation. So `std::bit_cast<std::uint16_t>(std::array<unsigned char, 2>{...})` works at compile time. Watch the endianness: that gives you the host's byte order, which is exactly the bug the shift version avoids."
>
> IF AHEAD: The slide's list says "non-literal types (pre-C++20)". The precise history: C++20 made far more types literal (constexpr destructors, which is what lets `std::string` and `std::vector` work), and C++23 relaxed the rule further so a `constexpr` function can declare a variable of non-literal type, as long as that path isn't evaluated at compile time. Similarly, `asm` blocks became allowed to appear in C++20 and `goto` in C++23, but only on paths that aren't evaluated. The rule underneath all of it: a `constexpr` function only has to work for the calls you actually make at compile time.
>
> IF BEHIND: "`reinterpret_cast` is never `constexpr`; that's why `as_bytes` isn't, and you'll hit it in task one."

Bridge: C++20 made the biggest jump. You can allocate.

A4. The rest of C++20 `constexpr` · C++20 (deck slide 50) (slow down)

The subtitle lists them: virtual calls, `try/catch` blocks, `dynamic_cast`, changing the active member of a union. All allowed in `constexpr` functions in C++20. A `throw` still can't be evaluated at compile time, but the `try` block is allowed to exist.

The code is about something more useful. **One function, two implementations: exact at compile time, fast at runtime.** Look at `power`. `if (std::is_constant_evaluated())`, the C++20 spelling, opens a branch that runs only during constant evaluation: a plain multiply loop. C++23 spells the same thing `if consteval`, and appendix slide 53 shows it. The `else` branch runs at runtime and calls `std::pow`, which is fast but not `constexpr` everywhere. The `static_assert` uses the compile-time branch.

(pause)

Now the trap, second block. Slow down here.

`std::is_constant_evaluated()` is the C++20 library function that answers "am I being evaluated at compile time right now?" Look inside `SHOW_ERRORS`: someone wrote `if constexpr (std::is_constant_evaluated())`. That looks right. It's wrong.

(pause)

Here's why. The condition of an `if constexpr` is itself a constant expression. It's evaluated at compile time, always. So when the compiler asks "am I being constant-evaluated?" while evaluating that condition, the answer is yes. Always. **`is_constant_evaluated()` inside `if constexpr` is always true, so the runtime branch is dead code.** GCC 14 warns about it.

The correct C++20 spelling is a plain `if`, line under the `#endif`. And the C++23 spelling is `if consteval`, which can't be misused that way, because there's no condition to put it in.

> DO: Run `./build/demos/s03/demo_s03_is_constant_evaluated`. Expected output: `1024 2`. Point at the 2: "`trap()` was called at runtime, so the plain `if` correctly took the runtime path." Then compile with `-DSHOW_ERRORS` and point at the GCC warning about `is_constant_evaluated` always evaluating to true in `if constexpr`. Recovery: "The output is `1024 2`; the warning text is in the comment on line 24."
>
> ASK: "If the compile-time branch is a multiply loop and the runtime branch is `std::pow`, are you guaranteed they give the same answer?" (pause) Expect: "no". Say: "Correct. Floating point at compile time and at runtime can differ, in the last bit or more, depending on the library, FMA contraction, and flags like fast-math. If you write two implementations, you need a test that runs both paths on the same inputs. That's a real review point."
>
> IF AHEAD: There's a mirror-image trap. `std::is_constant_evaluated()` called in a function that isn't `constexpr` at all is always false. And in a `constexpr` function called to initialize a non-`constexpr` local, it's false even if the optimizer later folds the call. "Constant-evaluated" means the language required a constant, not

that the compiler happened to compute it early. `if consteval` has the same semantics, minus the misuse. It's also needed to call a `consteval` function from inside a `constexpr` one: the `if consteval` branch is an immediate context, so the call is allowed there.

IF BEHIND: "`if consteval` gives one function two implementations; `is_constant_evaluated` inside `if constexpr` is always true, so don't."

Bridge: if `constexpr` means may, what's the keyword for must?

A5. `consteval` with a message · C++20 (deck slide 51)

This is the pattern for exercise task two, the config validator.

`Limit` has a name and a low and high bound. `validate` is `consteval` and returns a `std::string_view`. If the name is empty, return "limit has no name". If `lo` isn't less than `hi`, return "limit has lo >= hi". Otherwise return an empty view.

Then `static_assert(validate(kRpm).empty(), "sensor limit is invalid");`

(beat)

An empty string means valid; a non-empty string is the reason it failed. And because it's `consteval`, the validation can't accidentally become a runtime check that somebody disables in production.

Notice the test is `!(l.lo < l.hi)` and not `l.lo >= l.hi`. That's on purpose. If a bound is NaN, every comparison is false. `lo >= hi` would be false and the check would pass. `!(lo < hi)` is true and the check catches it. That's a habit worth having in any range validation.

(pause)

In the exercise, `validate` takes a `std::span<const SensorConfig>` and checks names, units, ranges, and duplicate names across the whole table, and the `static_assert` guards `kSensors`.

One honest caveat about the error you'll see. **The compiler prints your `static_assert` message and the failing expression. It does not print the string your validator returned.** So the message tells you the table is bad, and the code tells you which checks exist. C++26 lets you pass a computed string as the `static_assert` message, and then the reason prints directly.

IF AHEAD: Show the C++26 form if you preloaded it (Compiler Explorer tab 4): `static_assert(validate(kRpm).empty(), validate(kRpm));` on Clang 18 with `-std=c++2c`. Break `kRpm` by swapping the bounds. Clang prints "static assertion failed due to requirement 'validate(kRpm).empty()': limit has lo >= hi". That's P2741, user-generated `static_assert` messages; GCC 14 has it too in C++26 mode. Until your project is on C++26, a cheap trick to get the reason into the error is to make each failing check call a non-`constexpr` function, or `throw` a string literal, inside `validate`: the compiler then points at the exact line that failed. It's uglier, but the error lands on the line that names the problem.

IF BEHIND: "Return an empty `string_view` for valid and a reason for invalid, and `static_assert` on `.empty()`."

Bridge: that's compute-and-check. Now for globals you need to change at runtime.

A6. `constinit`: initialized at compile time, mutable at runtime · C++20 (deck slide 52)

There are three kinds of global, and C++20 finally named the one embedded code wants.

`constexpr` globals are immutable and constant-initialized. Plain globals are mutable and might be dynamically initialized, in an order you don't control across translation units. `constinit` is the middle one: mutable, and guaranteed constant-initialized.

Look at the code. `Counters` has two counters, `parsed` and `rejected`. `constinit Counters g_counters{};` That means the compiler guarantees `g_counters` is initialized before any code runs. It's in `.data` or `.bss` with its initial value. No constructor runs at startup.

(pause)

The commented line: `constinit int bad = fnv1a_runtime("x");` If the initializer isn't a constant expression, that's an error. That's the point. **`constinit` doesn't change what happens; it makes the compiler prove it.** Without it, someone changes an initializer, it silently becomes dynamic initialization, and now another translation unit's static constructor can read it before it's set. That's the static initialization order fiasco, and it's the kind of bug that shows up only when the link order changes.

`constinit` isn't `const`. The demo's `main` does `++g_counters.parsed`, and the output shows `1`.

> ASK: "Why not just make the counters `constexpr`?" Expect: "because they need to change". Say: "Right. `constexpr` implies `const`. `constinit` is for exactly the globals that must change but must also be ready before anything runs: counters, state machines, error flags."
>
> IF AHEAD: `constinit` pairs well with `thread_local`. A `thread_local` variable with dynamic initialization makes every access go through a wrapper function that checks initialization. If you declare it `extern thread_local constinit int x;` in the header, every translation unit knows there's no dynamic initializer, and the compiler can access it directly. That's a measurable win in hot paths. Also, `constinit` applies only to variables with static or thread storage duration; on a local it's an error.
>
> IF BEHIND: "`constinit`: mutable global, initializer proven constant, no initialization-order fiasco."

Bridge: C++23 cleaned up nearly everything else.

A7. Almost nothing left · C++23 (deck slide 53)

C++23 removed most of the remaining restrictions. The subtitle: `if consteval`, static locals, non-literal variables, `constexpr std::unique_ptr`.

The code shows static locals. `cached_or_computed` has `if consteval` on top. At compile time, just compute the factorial. At runtime, go to the `else` branch, which has a `static std::array<int, 13> cache`, and memoize. Before C++23, a `static` local anywhere in a `constexpr` function was an error, even on a branch that couldn't run at compile time. C++23 allows it, as long as the compile-time path doesn't touch it.

(pause)

The bigger C++23 change is in the notes: a function may be `constexpr` as long as some call could be a constant expression. The old rule required every path to be constant-evaluable. Now the compiler only checks what you actually evaluate.

A code-review point on that example: the runtime cache is a function-local static that's written without synchronization. Call it from two threads and that's a data race. Good for a slide; in production, it needs an atomic or a lock, or it shouldn't be a cache.

The bottom of the slide is what's still not `constexpr` in C++23: throwing exceptions, placement `new`, most of `<cmath>`, and `reinterpret_cast`, which is never. Throwing at compile time and placement `new` are C++26. Some of `<cmath>` became `constexpr` in C++23 (things like `fabs`, `floor`, `fmin`) and C++26 adds most of the rest.

> IF AHEAD: The table size 13 isn't random. 12 factorial is 479,001,600, which fits in a 32-bit `int`. 13 factorial is about 6.2 billion, which doesn't. So the cache covers 0 through 12 and the next index would overflow. That's a nice way to tie back to appendix slide 48: `static_assert(cached_or_computed(13) > 0)` would fail the build with an overflow error, and the runtime call would just be UB. Also from this slide's subtitle: `constexpr std::unique_ptr` (C++23) means RAII code can run at compile time, as long as the allocation is freed, same as the vector rule.
>
> IF BEHIND: "C++23 allows static locals and non-literal variables on runtime-only paths; almost nothing is left that can't be `constexpr`."

Bridge: here's the whole story on one slide.

A8. The full timeline (deck slide 54)

Twelve years from a glorified macro to nearly the whole language at compile time.

C++11: single return, recursion. C++14: loops and locals, which is where the CRC loop becomes legal. C++17: lambdas, `if constexpr`, `std::array`, `inline constexpr` variables, which is where the CRC table lives. C++20: allocation, `vector` and `string`, `consteval`, `constinit`, which is the config validator. C++23: `if consteval`, static locals, and the relaxed rules that let the exercise's lookup functions move into headers. C++26: exceptions, placement `new`, and `static_assert` with a computed message.

(beat)

Point at where each exercise function sits: every row has a piece of today's code on it. And notice how much of this works on a C++17 compiler. If your certified toolchain is stuck at C++17, you still get the CRC table and most of the validation pattern, minus `consteval`.

IF AHEAD: For the record on the C++23 row: `string_view` has been usable in constant expressions since C++17, and `std::optional` became fully `constexpr` in C++20 (P2231 was applied as a defect report against C++20). So `find_sensor` returning an `optional` in a `static_assert` doesn't actually need C++23; it needs a library new enough to have implemented the C++20 fixes.

IF BEHIND: "Every standard expanded `constexpr`; the exercise functions sit on the C++14, 17, and 20 rows."

Bridge: the takeaway.

A9. Template lambdas and unevaluated lambdas · C++20 (deck slide 55)

Top block. `sum14` is a C++14 generic lambda: `const auto& s`. It works, but inside the body you don't have a name for the element type. So the accumulator is a `double`, which is wrong for integers.

`sum20` is a C++20 template lambda: `[]<typename T>(std::span<const T> s)`. Now `T` is a name. The accumulator is `T t{}`. **Template lambdas let you name the parameter types, constrain them, and use them in the body.**

(pause)

Bottom block, the practical one. In C++20 a lambda can appear in an unevaluated context, like `decltype`, and captureless lambdas are default-constructible. Two consequences.

`std::set<std::string, decltype(less_by_size)>`: a set with a lambda comparator and no named comparator struct.

And the line I want you to remember: `std::unique_ptr<FILE, decltype([](FILE* f) { if (f) std::fclose(f); })>`. **That's a RAII wrapper for any C handle in one declaration: no named deleter struct, no `std::function`, and no size overhead, because the deleter is an empty type.** Every embedded codebase has C APIs with open and close pairs. This is how you wrap them.

DO: Run `./build/demos/s03/demo_s03_template_lambdas`. Expected output: `6 6 a true`. Point at `a`: "the set is ordered by length, so the one-character string is first." Recovery: "Six, six, a, true."

IF AHEAD: One caution on lambdas in `decltype`. Every lambda expression has a unique type, and that includes a lambda written in a header and included into two translation units. Inside a function body or a `.cpp`, which is how the demo uses it, that's fine. In a namespace-scope type alias in a header that's used in function signatures across translation units, you're close to ODR trouble. For a deleter you'll reuse across files, a named struct with `operator()` is still the safe choice. Also, the demo file uses `FILE` and `fopen` without including `<cstdio>`; it compiles through transitive includes, but your code should include it.

IF BEHIND: "Template lambdas name their types; `decltype` of a lambda gives you a one-line RAII deleter."

Bridge: values as template arguments, part two.

A10. Class types as template arguments · C++20 (deck slide 56)

C++20 lets a class type be a non-type template parameter, if it's structural. **Structural means all members public, none mutable, and every member itself structural.** Scalars, arrays of structural types, and classes made of them.

`FixedString` holds a `char` array of size `N` and copies a string literal into it. It's structural. So `template <FixedString Name> struct Sensor` takes a string literal as a template argument. `Sensor<"rpm">::name == "rpm"`. A compile-time string, as part of a type.

(pause)

Why can't the exercise's `SensorConfig` be a template argument? It has `string_view` members. `std::string_view` keeps its pointer and length as private members. Private members mean not structural. So any struct holding a `string_view` can't be an NTTP. If you replaced the `string_view` with a `FixedString`, it could be.

The `FixedString` idiom is in every modern library that wants compile-time names: format string checking, reflection-like registries, compile-time routing.

> DO: Optional, if not shown on slide 20: `./build/demos/s03/demo_s03_auto_nttp` prints `7 temp_core`. Point at `Sensor<"temp_core">::name`.
>
> IF AHEAD: Floating-point values became allowed as NTTPs in C++20 too, compared by bit pattern, so `0.0` and `-0.0` are different template arguments and NaN payloads matter. And two `Sensor<"rpm">` written in different files are the same type, because template-argument equivalence for class types compares member by member. That's what makes a compile-time registry keyed by name work across translation units. Note that `FixedString`'s constructor copies `N` characters including the null; `view()` drops it with `N - 1`.
>
> IF BEHIND: "Structural class types can be template arguments; `string_view` can't, which is why `SensorConfig` can't."

Bridge: back to error messages, one more time, side by side.

Running the labs

The per-lab coaching is in the script at slides 13, 22, 35 and 43. The rhythm is the same every time:

1. Lab slide up, starter link in chat, one quiet minute while people open the file. Then stay quiet for five more; answer chat, do not lecture.
2. At seven minutes, paste the solution link and walk the three changes in two minutes. The debrief is the last word on the block's topic, so it is scripted.
3. One question to the room, then move on at ten minutes whether or not people are done. Nothing later depends on finishing a lab.

If the lecture runs late, the lab still starts within two minutes of its target; cut the slide before the takeaway instead. Block 2 is scripted for fifteen minutes so that block 1, the longest story in

the course, has five minutes of slack before block 3 starts at 1:00. If a lab runs over because the room is engaged, that is the time to spend; block 4 is short and has no slack of its own.

The at-home exercise

Launched at slide 44 in one minute; nothing is done in class. What to say when it comes up in chat during the week, and in next session's recap:

- Task 1, the `as_bytes` snag. This is deliberate. The fix is to write the CRC core over `std::span<const Byte>` for any one-byte `Byte` with `static_cast<unsigned char>` per element, and have the `string_view` overload forward to it. Appendix slide 49 is the explanation: `reinterpret_cast` is never a constant expression. (Lab 3.1 TODO 1 was the rehearsal for the rest of the task.)
- Task 1, forgetting `inline` on `kCrcTable` in the header, or leaving the old function-local static in `crc.cpp` and getting a duplicate definition.
- Task 1, `crc16` itself not `constexpr`, so the `static_assert` fails with "call to non-`constexpr` function".
- Task 2, `kSensors` still defined in `config.cpp`. The `static_assert` needs it visible in the header as `inline constexpr std::array<SensorConfig, 6>`. Lab 3.1 TODO 2 showed the same error in miniature: a `const` array of class type is not usable in a constant expression.
- Task 2, writing `min_valid >= max_valid` instead of `!(min_valid < max_valid)`. Both pass on the real table; only one catches NaN. Mention it if you see it, don't block on it.
- Task 2, duplicate-name check: a nested loop over the span is fine and `constexpr`. People reach for `std::set` or `std::unordered_set`; at compile time a `std::vector` plus `std::sort` and `std::adjacent_find` works, or just the nested loop.
- Task 2, the error: only the `static_assert` message and the failed expression print, not the returned reason (GCC 13, Clang 18). Keep the checks small and obvious so the reader of a failed build can find the cause.
- Task 3, `bool` and `char` matching `std::integral auto`. If a test expects `serialize('x')` to be a string, the integral overload grabbed it. Check what the tests require.
- Task 3, writing `requires !StringLike<T>` without parentheses in a requires-clause (GCC: "expression must be enclosed in parentheses").
- Task 3, `serialize(std::string)` becoming ambiguous if anyone jumps ahead to a range overload without the `!StringLike` exclusion. That's task four; slide 29 and lab 3.3's TODO 3 explain it.
- Task 7: `this Self&& self` with no `this` pointer in the body; `self.count`, `self.sum`. Lab 3.4 TODO 1 and TODO 3 were the rehearsal.
- Anyone on Clang 18 with libstdc++ instead of libc++ will have trouble with `std::expected` from Session 2; the repo's CMake adds `-stdlib=libc++` for Clang, so it's only people who changed that.

Next session's recap slide should pick up whichever of these the chat shows most.

Likely questions and answers

Our certified toolchain is stuck on C++17. What from today can we use? Most of block one and all of block two: relaxed `constexpr`, `constexpr` lambdas, `std::array` tables, `inline constexpr`, `static_assert` on table contents, fold expressions, `if constexpr`, CTAD, `auto` NTTPs. You lose `consteval`, `constinit`, compile-time allocation, concepts, and deducing `this`. The `validate` pattern works with plain `constexpr`; you just lose the guarantee that nobody calls it at runtime.

Which compiler versions do we need for concepts and deducing `this`? The course baseline is GCC 14 and Clang 18, and everything today is verified on both per the toolchain matrix. Roughly: concepts are usable from GCC 10 and Clang 10, with Clang's support maturing over the next several releases; deducing `this` needs GCC 14, Clang 18, or MSVC from VS 2022 17.2. Check your exact version on Compiler Explorer before you commit to a feature.

Does putting `constexpr` everywhere slow down builds? Only where the compiler actually evaluates something at compile time, and only in proportion to the work. A 256-entry table is unmeasurable. A large table, or a heavy `consteval` in a header included everywhere, can show up; Clang's `-ftime-trace` tells you where compile time goes. The compilers cap evaluation with `-fconstexpr-ops-limit` and `-fconstexpr-loop-limit` (GCC) and `-fconstexpr-steps` (Clang) so a runaway loop fails instead of hanging.

How do I debug a `constexpr` function? I can't step through the compiler. You can't step through constant evaluation, but a `constexpr` function is also an ordinary function. Call it at runtime from a unit test with the same inputs and debug that. The `static_assert` tells you that something is wrong; the runtime test tells you where. That's one reason to prefer `constexpr` over `consteval` unless you need the guarantee.

What do MISRA and AUTOSAR say about this? AUTOSAR C++14 has a rule encouraging `constexpr` for values that can be determined at compile time, and MISRA C++:2023 targets C++17, so concepts, `consteval`, `constinit`, and deducing `this` are outside its scope rather than prohibited. Both standards dislike recursion, which is relevant to C++11-style `constexpr` and recursive lambdas. Check your program's deviation process before adopting post-C++17 features in certified code.

Do concepts change the ABI or the generated code? Concepts are checked at compile time and generate no code of their own; a constrained template instantiates to the same machine code as the unconstrained one. What can change is overload resolution: adding a constraint, or a more-constrained overload, can change which function a call picks, which is a source-compatibility issue. Deducing `this` turns a non-template member into a template, which moves it into the header and changes exported symbols, so treat that as an ABI change for shared libraries.

Why not just put a `static_assert` inside the template body instead of a concept? A `static_assert` inside the body fires after overload resolution has already picked the function, so it can't steer the choice to another overload, and `if constexpr (Concept<T>)` or SFINAE-style checks can't see it. A concept participates in overload resolution and appears in the signature. Use `static_assert` for "this can never be right", concepts for "this overload isn't the right one".

Can compile-time and runtime results differ, for example in floating point? Yes. The compiler evaluates floating point at compile time with its own arithmetic, and runtime results depend on the library, FMA contraction, `-ffast-math`, and the target. If you use `if consteval` to provide two implementations, test both paths on the same inputs. For integer tables like the CRC, the results are exact and identical.

Will compile-time tables bloat our flash? Usually the opposite. The table goes into `.rodata`, which on most microcontrollers is flash, and the code that built the table at runtime is gone, as is the RAM copy. What can grow is template instantiations: every distinct `Concept auto` argument type is another function. That's the same trade as any template.

GCC's concept error just says "constraints not satisfied". How do I see which part failed? Add `-fconcepts-diagnostics-depth=2` or 3 to GCC to expand nested concepts in the error. Clang shows more of the nesting by default. And building concepts from small named pieces, as on slide 30, makes the failing piece name itself.

Is `consteval` contagious? I get "not a constant expression" calling it from a `constexpr` function. In C++20 as published, yes: calling a `consteval` function with a parameter from a `constexpr` function is an error. C++23's P2564, applied as a defect report, makes the enclosing `constexpr` function implicitly `consteval` instead; GCC 14 and recent Clang implement it. On older compilers, call the `consteval` function inside an `if consteval` branch, or make the caller `consteval`.

Can an explicit-object member function be virtual? No. Explicit-object member functions can't be `virtual` or `static`, and they can't have `const` or ref-qualifiers after the parameter list, because the object parameter already expresses those. If you need runtime polymorphism, use a regular virtual function; deducing `this` is for static dispatch and value-category forwarding.

Deck issues found

Slide numbers in this list are the deck's numbering before the block restructure (see `docs/session-format.md`); the items are kept for the record.

Status: fixed in the deck, demos, outline and README on 2026-10-02 (see scripts/README.md). Items kept for the record. Still open: the Compiler Explorer `<add short link>` placeholders in every demo file header.

- Slides 21 and 29: say "Seven candidates", but the quoted error lists eight (seven overloads plus the `span` template); the README says "seven overloads plus a template".
- Slides 4 and 49: "four constrained templates" in the solution, but README tasks 3 and 4 describe five (`integral`, `floating_point`, `StringLike`, `SerializableRange`, and the `Status/Record` overload). Verify against `solution/`'s `serialize.h`.
- Slide 4: a function-local static is initialized on first call, not "at startup"; the "one more thing at startup" wording contradicts the README ("at first use").
- Slide 12: "non-literal types (pre-C++20)" conflicts with slide 18; non-literal variables in `constexpr` functions came with C++23 (P2242). Also `asm` (C++20) and `goto` (C++23) may appear in `constexpr` functions; only evaluating them is refused.

- Slide 16 notes: claim the returned `string_view` appears in an "evaluated to" note. Checked on GCC 13 and Clang 18: only the `static_assert` message and the expression print. Clang 18 in `-std=c++2c` with the validator's result as the message (P2741) does print the reason.
- Slides 12 and 18 (and `constexpr_evolution.cpp` line 40): "`std::to_string` (C++26)". I couldn't confirm that a C++26 paper makes `std::to_string` `constexpr`; verify. Integral `std::to_chars` is `constexpr` since C++23 and is the safe workaround to cite.
- Slide 19: "`optional/string_view` everywhere" listed under C++23, but `string_view` has been usable in constant expressions since C++17, and `std::optional` became fully `constexpr` in C++20 (P2231, applied as a DR).
- Slide 25 (and outline item 22): the code comment labels `std::remove_const_t` as C++17; the `_t` aliases are C++14 (N3655), and only the `_v` aliases are C++17.
- Slide 25 / `variable_templates.cpp`: `static_assert(!is_small_v<long double>)` fails wherever `long double` is 8 bytes (Apple Silicon including Homebrew GCC on arm64 macOS, MSVC). Linux x86-64 CI won't catch it.
- Slides 29 and 49: both run the same starter-versus-solution `serialize(ParseError)` comparison live; the script treats slide 49 as a ten-second reprise plus the table break.
- Slide 32: the "Before (C++11)" column uses `std::string_view`, which is C++17.
- Slide 34: `std::three_way_comparable` is declared in `<compare>`, not `<concepts>`.
- Slide 35 notes versus README task 4: the README says that without `!StringLike` a `std::string` "would serialize as a list of chars"; with both overloads viable and neither subsuming, the call is actually ambiguous, as the deck says.
- Slide 38 notes: say the "weakest concept" position matches Core Guidelines T.10 to T.26, but T.20 and T.21 argue against minimal syntactic concepts; "weakest meaningful concept" is closer.
- Slide 46 notes: "a plain struct you can put in a container" is misleading; calling `draw()` through a `Shape&` deduces `Shape` and doesn't compile. It's still static dispatch only.
- Slide 14: badge is C++20, but the headline example uses C++23 `if consteval`.
- Slide 44: the call comments (`stats.add(1).add(2)`) leave out the `Status` argument that the signature requires.
- `template_lambdas.cpp` uses `FILE`, `std::fopen`, and `std::fclose` without `#include <cstdio>`; it compiles only through transitive includes.
- All `demos/s03/*.cpp` headers still say `Compiler Explorer: <add short link>`.
- Outline: the header says "54 content slides", but the outline lists 48 and the deck has 51. Outline item 37 (`constexpr` + concepts registry) is cut, and the outline's demo list names `class_nttp.cpp`, which doesn't exist (FixedString is in `auto_nttp.cpp`).
- Slide 2: the agenda totals 115 minutes and doesn't list the 5-minute wrap-up. Slide 49's 2 minutes plus the 20-minute exercise end at 1:57, which leaves 3 minutes for slide 51.